

订单系统详细设计

一、数据库设计

正向订单表

订单表 (order_info) :

订单条目表 (order_item) :

订单支付明细表 (order_payment_detail)

订单费用表 (order_amount)

订单费用明细表 (order_amount_detail)

订单快照表 (order_snapshot)

订单配送信息表 (order_delivery_detail)

订单操作日志表 (order_operate_log)

逆向售后表

after_sale_info(订单售后表)

after_sale_item (订单售后条目表)

after_sale_refund (售后退款表)

after_sale_log (售后单变更表)

正向订单相关枚举

订单类型 (orderType)

业务线 (businessIdentifier)

订单状态 (orderStatus)

支付类型 (payType)

支付状态 (payStatus)

费用类型 (amountType)

账户类型 (accountType)

配送方式 (deliveryType)

订单取消类型 (orderCancelType)

平台 (platForm)

商品类型 (productType)

订单查询类型 (QueryDataType)

逆向售后相关枚举

售后申请来源枚举

售后申请原因枚举

售后明细类型

售后支付状态枚举 (AfterSalePaymentStatus)

售后单状态枚举 (AfterSaleStatus)

售后类型 (AfterSaleType)

二、接口设计

1、正向订单业务

1.1、生成订单号接口

接口定义：

请求入参：

返回结果：

使用场景：

具体实现：

1.2、提交订单接口

接口定义：

请求入参：

返回结果：

使用场景：

具体实现：

技术要点：

1.3、预支付订单接口

接口定义：

请求入参：

返回结果：

使用场景：

具体实现：

技术要点：

1.4、支付回调接口

接口定义：

请求入参：

返回结果:

使用场景:

具体实现:

技术要点:

1.5、触发订单履约流程接口

所属系统: 订单系统(ryuan-eshop-order-service)

调用时机

实现方法

接口定义

请求入参

返回结果

业务规则

业务流程

技术要点

1.6、接受订单履约接口

所属系统: 履约系统(ryuan-eshop-fulfill-service)

接口说明

调用时机

实现方法

接口定义

请求入参

返回结果

业务流程

1.7、触发订单物流配送结果事件接口

所属系统: 履约系统(ryuan-eshop-fulfill-service)

接口说明

实现方法

接口定义

请求入参

返回结果

业务流程

1.8、通知订单物流配送结果接口

所属系统: 订单系统(ryuan-eshop-order-service)

调用时机

实现方法

接口定义

请求参数

返回结果

业务流程

技术要点

实现亮点:

1.9、删除订单

所属系统: 订单系统(ryuan-eshop-order-service)

实现方法

接口定义

请求参数

返回结果

业务规则

具体实现

1.10、修改配送地址

所属系统: 订单系统(ryuan-eshop-order-service)

实现方法

接口定义

请求参数

返回结果

业务规则

具体实现

1.11、查询订单列表

所属系统: 订单系统(ryuan-eshop-order-service)

实现方法

接口定义

请求参数

返回结果

业务规则

字段映射梳理

具体实现：

实现亮点：

二期功能实现

1.12、查询订单详情

所属系统：订单系统(ruyuan-eshop-order-service)

实现方法

接口定义

请求参数

返回结果

业务规则

字段映射梳理

具体实现

2、逆向售后业务

2.1、取消订单/超时未支付取消 接口

接口定义：

请求入参：

返回结果：

使用场景：

具体实现：

技术要点：

2.2、支付退款回调 接口

接口定义：

请求入参：

返回结果：

使用场景：

具体实现：

2.3、处理售后申请 接口

接口定义：

请求入参：

返回结果：

使用场景：

具体实现：

2.4、客服售后审核 接口

接口定义：

请求入参：

返回结果：

使用场景：

具体实现：

2.5、缺品接口

所属系统：订单系统(ryuan-eshop-order-service)

实现方法

接口定义

请求参数

返回结果

使用场景

业务规则

具体实现

技术要点

2.7、用户撤销售后申请

所属系统：订单系统(ryuan-eshop-order-service)

实现方法

接口定义

请求参数

返回结果

业务规则

具体实现

技术要点

2.8、自动取消超时订单任务

所属系统：订单系统(ryuan-eshop-order-service)

实现方法

使用场景

具体实现

技术要点

2.9、查询售后列表

所属系统：订单系统(ruyuan-eshop-order-service)

实现方法

接口定义

请求参数

返回结果

业务规则

字段映射梳理

具体实现

2.10、查询售后详情

所属系统：订单系统(ruyuan-eshop-order-service)

实现方法

接口定义

请求参数

返回结果

字段映射梳理

具体实现

一、数据库设计

正向订单表

订单表（order_info）：

```

1 CREATE TABLE `order_info` (
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键ID',
3   `business_identifier` tinyint(4) NOT NULL COMMENT '接入方业务线标识 1, "自
  营商城"',
4   `order_id` varchar(50) NOT NULL COMMENT '订单编号',
5   `parent_order_id` varchar(50) DEFAULT NULL COMMENT '父订单编号',
6   `business_order_id` varchar(50) DEFAULT NULL COMMENT '接入方订单号',
7   `order_type` tinyint(4) NOT NULL COMMENT '订单类型 1:一般订单 255:其它',
8   `order_status` tinyint(4) NOT NULL COMMENT '订单状态 10:已创建, 30:已履约,
  40:出库, 50:配送中, 60:已签收, 70:已取消, 100:已拒收, 255:无效订单',
9   `cancel_type` varchar(255) DEFAULT NULL COMMENT '订单取消类型',
10  `cancel_time` datetime DEFAULT NULL COMMENT '订单取消时间',
11  `seller_id` varchar(50) DEFAULT NULL COMMENT '卖家编号',
12  `user_id` varchar(50) DEFAULT NULL COMMENT '买家编号',
13  `total_amount` int(11) DEFAULT NULL COMMENT '交易总金额 (以分为单位存储)',
14  `pay_amount` int(11) DEFAULT NULL COMMENT '交易支付金额',
15  `pay_type` tinyint(4) DEFAULT NULL COMMENT '支付方式 10:微信支付 20:支付宝支
  付',
16  `coupon_id` varchar(50) DEFAULT NULL COMMENT '使用的优惠券编号',
17  `pay_time` datetime DEFAULT NULL COMMENT '支付时间',
18  `expire_time` datetime NOT NULL COMMENT '支付订单截止时间',
19  `user_remark` varchar(255) DEFAULT NULL COMMENT '用户备注',
20  `delete_status` tinyint(4) NOT NULL COMMENT '订单删除状态 0:未删除 1:已删
  除',
21  `comment_status` tinyint(4) NOT NULL DEFAULT '0' COMMENT '订单评论状态 0:未
  发表评论 1:已发表评论',
22  `ext_json` varchar(1024) DEFAULT NULL COMMENT '扩展信息',
23  `gmt_create` datetime NOT NULL COMMENT '创建时间',
24  `gmt_modified` datetime NOT NULL COMMENT '更新时间',
25  PRIMARY KEY (`id`) USING BTREE,
26  UNIQUE KEY `idx_order_id` (`order_id`) USING BTREE
27 ) ENGINE=InnoDB AUTO_INCREMENT=7 DEFAULT CHARSET=utf8 COMMENT='订单表';

```

订单条目表 (order_item) :


```

1 CREATE TABLE `order_item` (
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键ID',
3   `order_id` varchar(50) NOT NULL COMMENT '订单编号',
4   `order_item_id` varchar(50) NOT NULL COMMENT '订单明细编号',
5   `product_type` tinyint(4) NOT NULL COMMENT '商品类型 1:普通商品,2:预售商品',
6   `product_id` varchar(50) NOT NULL COMMENT '商品编号',
7   `product_img` varchar(255) DEFAULT NULL COMMENT '商品图片',
8   `product_name` varchar(50) NOT NULL COMMENT '商品名称',
9   `sku_code` varchar(50) NOT NULL COMMENT 'sku编码',
10  `sale_quantity` int(11) NOT NULL COMMENT '销售数量',
11  `sale_price` int(11) NOT NULL COMMENT '销售单价',
12  `origin_amount` int(11) NOT NULL COMMENT '当前商品支付原总价',
13  `pay_amount` int(11) NOT NULL COMMENT '交易支付金额',
14  `product_unit` varchar(10) NOT NULL COMMENT '商品单位',
15  `purchase_price` int(11) DEFAULT NULL COMMENT '采购成本价',
16  `seller_id` varchar(50) DEFAULT NULL COMMENT '卖家编号',
17  `gmt_create` datetime NOT NULL COMMENT '创建时间',
18  `gmt_modified` datetime NOT NULL COMMENT '更新时间',
19  PRIMARY KEY (`id`) USING BTREE,
20  KEY `idx_order_id` (`order_id`) USING BTREE
21 ) ENGINE=InnoDB AUTO_INCREMENT=10 DEFAULT CHARSET=utf8 COMMENT='订单条目表';

```

订单支付明细表 (order_payment_detail)

```

1 CREATE TABLE `order_payment_detail` (
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键ID',
3   `order_id` varchar(50) NOT NULL COMMENT '订单编号',
4   `account_type` tinyint(4) NOT NULL COMMENT '账户类型',
5   `pay_type` tinyint(4) NOT NULL COMMENT '支付类型 10:微信支付, 20:支付宝支付',
6   `pay_status` tinyint(4) NOT NULL COMMENT '支付状态 10:未支付,20:已支付',
7   `pay_amount` int(11) NOT NULL DEFAULT '0' COMMENT '支付金额',
8   `pay_time` datetime DEFAULT NULL COMMENT '支付时间',
9   `out_trade_no` varchar(50) DEFAULT NULL COMMENT '支付系统交易流水号',
10  `pay_remark` varchar(255) DEFAULT NULL COMMENT '支付备注信息',
11  `gmt_create` datetime NOT NULL COMMENT '创建时间',
12  `gmt_modified` datetime NOT NULL COMMENT '更新时间',
13  PRIMARY KEY (`id`) USING BTREE,
14  KEY `idx_order_id` (`order_id`) USING BTREE
15 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COMMENT='订单支付明细表';

```

订单费用表 (order_amount)

SQL [复制代码](#)

```
1 CREATE TABLE `order_amount` (  
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键ID',  
3   `order_id` varchar(50) NOT NULL COMMENT '订单编号',  
4   `amount_type` int(11) NOT NULL COMMENT '收费类型',  
5   `amount` int(11) NOT NULL COMMENT '收费金额',  
6   `gmt_create` datetime NOT NULL COMMENT '创建时间',  
7   `gmt_modified` datetime NOT NULL COMMENT '更新时间',  
8   PRIMARY KEY (`id`) USING BTREE  
9 ) ENGINE=InnoDB AUTO_INCREMENT=21 DEFAULT CHARSET=utf8 COMMENT='订单费用表';
```

订单费用明细表 (order_amount_detail)

SQL [复制代码](#)

```
1 CREATE TABLE `order_amount_detail` (  
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键ID',  
3   `order_id` varchar(50) NOT NULL COMMENT '订单编号',  
4   `product_type` tinyint(4) NOT NULL COMMENT '产品类型',  
5   `order_item_id` varchar(50) NOT NULL COMMENT '订单明细编号',  
6   `product_id` varchar(50) NOT NULL COMMENT '商品编号',  
7   `sku_code` varchar(50) NOT NULL COMMENT 'sku编码',  
8   `sale_quantity` int(11) NOT NULL COMMENT '销售数量',  
9   `sale_price` int(11) NOT NULL COMMENT '销售单价',  
10  `amount_type` tinyint(4) NOT NULL COMMENT '收费类型',  
11  `amount` int(11) NOT NULL COMMENT '收费金额',  
12  `gmt_create` datetime NOT NULL COMMENT '创建时间',  
13  `gmt_modified` datetime NOT NULL COMMENT '更新时间',  
14  PRIMARY KEY (`id`) USING BTREE  
15 ) ENGINE=InnoDB AUTO_INCREMENT=28 DEFAULT CHARSET=utf8 COMMENT='订单费用明细表';
```

订单快照表 (order_snapshot)

```

1 CREATE TABLE `order_snapshot` (
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键ID',
3   `order_id` varchar(50) NOT NULL COMMENT '订单号',
4   `snapshot_type` tinyint(4) unsigned NOT NULL COMMENT '快照类型',
5   `snapshot_json` varchar(2000) NOT NULL COMMENT '订单快照内容',
6   `gmt_create` datetime NOT NULL COMMENT '创建时间',
7   `gmt_modified` datetime NOT NULL COMMENT '更新时间',
8   PRIMARY KEY (`id`) USING BTREE,
9   KEY `idx_order_id` (`order_id`) USING BTREE
10  ) ENGINE=InnoDB AUTO_INCREMENT=3 DEFAULT CHARSET=utf8 COMMENT='订单快照表';

```

订单配送信息表 (order_delivery_detail)

```

1 CREATE TABLE `order_delivery_detail` (
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键ID',
3   `order_id` varchar(50) NOT NULL COMMENT '订单编号',
4   `delivery_type` tinyint(4) DEFAULT NULL COMMENT '配送类型',
5   `province` varchar(50) DEFAULT NULL COMMENT '省',
6   `city` varchar(50) DEFAULT NULL COMMENT '市',
7   `area` varchar(50) DEFAULT NULL COMMENT '区',
8   `street` varchar(50) DEFAULT NULL COMMENT '街道',
9   `detail_address` varchar(255) DEFAULT NULL COMMENT '详细地址',
10  `lon` decimal(20,10) DEFAULT NULL COMMENT '经度',
11  `lat` decimal(20,10) DEFAULT NULL COMMENT '纬度',
12  `receiver_name` varchar(50) DEFAULT NULL COMMENT '收货人姓名',
13  `receiver_phone` varchar(50) DEFAULT NULL COMMENT '收货人电话',
14  `modify_address_count` tinyint(4) NOT NULL DEFAULT '0' COMMENT '调整地址次数',
15  `deliverer_no` varchar(50) DEFAULT NULL COMMENT '配送员编号',
16  `deliverer_name` varchar(50) DEFAULT NULL COMMENT '配送员姓名',
17  `deliverer_phone` varchar(50) DEFAULT NULL COMMENT '配送员手机号',
18  `out_stock_time` datetime DEFAULT NULL COMMENT '出库时间',
19  `signed_time` datetime DEFAULT NULL COMMENT '签收时间',
20  `gmt_create` datetime NOT NULL COMMENT '创建时间',
21  `gmt_modified` datetime NOT NULL COMMENT '更新时间',
22  PRIMARY KEY (`id`) USING BTREE
23  ) ENGINE=InnoDB AUTO_INCREMENT=6 DEFAULT CHARSET=utf8 COMMENT='订单配送信息表';

```

订单操作日志表 (order_operate_log)

SQL | 复制代码

```
1 CREATE TABLE `order_operate_log` (  
2   `id` bigint(20) NOT NULL COMMENT '主键ID',  
3   `order_id` varchar(50) NOT NULL COMMENT '订单编号',  
4   `operate_type` tinyint(4) DEFAULT NULL COMMENT '操作类型',  
5   `pre_status` tinyint(4) DEFAULT NULL COMMENT '前置状态',  
6   `current_status` tinyint(4) DEFAULT NULL COMMENT '当前状态',  
7   `remark` varchar(255) DEFAULT NULL COMMENT '备注说明',  
8   `gmt_create` datetime NOT NULL COMMENT '创建时间',  
9   `gmt_modified` datetime NOT NULL COMMENT '更新时间',  
10  PRIMARY KEY (`id`) USING BTREE,  
11  KEY `idx_order_id` (`order_id`) USING BTREE  
12 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COMMENT='订单操作日志表';
```

逆向售后表

after_sale_info(订单售后表)

```

1 CREATE TABLE `after_sale_info` (
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键id',
3   `business_identifier` tinyint(4) DEFAULT NULL COMMENT '接入方业务线标识 1,
  "自营商城"',
4   `after_sale_id` bigint(20) NOT NULL COMMENT '售后id',
5   `order_id` varchar(50) NOT NULL DEFAULT '' COMMENT '订单号',
6   `order_source_channel` tinyint(4) NOT NULL COMMENT '订单来源渠道',
7   `user_id` varchar(20) NOT NULL COMMENT '购买用户id',
8   `after_sale_status` tinyint(4) NOT NULL COMMENT '售后单状态',
9   `order_type` tinyint(4) NOT NULL COMMENT '订单类型',
10  `apply_source` tinyint(4) DEFAULT NULL COMMENT '申请售后来源',
11  `apply_time` datetime DEFAULT NULL COMMENT '申请售后时间',
12  `apply_reason_code` tinyint(4) DEFAULT NULL COMMENT '申请原因编码',
13  `apply_reason` varchar(1024) DEFAULT NULL COMMENT '申请原因',
14  `review_time` datetime DEFAULT NULL COMMENT '客服审核时间',
15  `review_source` tinyint(4) DEFAULT NULL COMMENT '客服审核来源',
16  `review_reason_code` tinyint(4) DEFAULT NULL COMMENT '客服审核结果编码',
17  `review_reason` varchar(1024) DEFAULT NULL COMMENT '客服审核结果',
18  `after_sale_type` tinyint(4) NOT NULL COMMENT '售后类型',
19  `after_sale_type_detail` tinyint(4) DEFAULT NULL COMMENT '售后类型详情枚举',
20  `apply_refund_amount` int(11) DEFAULT NULL COMMENT '申请退款金额',
21  `real_refund_amount` int(11) DEFAULT NULL COMMENT '实际退款金额',
22  `remark` varchar(1024) DEFAULT NULL COMMENT '备注',
23  `gmt_create` datetime NOT NULL COMMENT '创建时间',
24  `gmt_modified` datetime NOT NULL COMMENT '更新时间',
25  PRIMARY KEY (`id`),
26  KEY `idx_after_sale_type` (`after_sale_type`),
27  KEY `idx_order_id` (`order_id`)
28 ) ENGINE=InnoDB AUTO_INCREMENT=125 DEFAULT CHARSET=utf8 COMMENT='订单售后表';

```

after_sale_item (订单售后条目表)

```

1 CREATE TABLE `after_sale_item` (
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键id',
3   `after_sale_id` bigint(20) NOT NULL COMMENT '售后id',
4   `order_id` varchar(50) NOT NULL DEFAULT '' COMMENT '订单id',
5   `sku_code` varchar(20) NOT NULL DEFAULT '' COMMENT 'sku id',
6   `product_name` varchar(1024) NOT NULL COMMENT '商品名',
7   `return_quantity` tinyint(4) NOT NULL COMMENT '商品退货数量',
8   `product_img` varchar(1024) NOT NULL COMMENT '商品图片地址',
9   `origin_amount` int(11) NOT NULL COMMENT '商品总金额',
10  `apply_refund_amount` int(11) NOT NULL COMMENT '申请退款金额',
11  `real_refund_amount` int(11) NOT NULL COMMENT '实际退款金额',
12  `gmt_create` datetime NOT NULL COMMENT '创建时间',
13  `gmt_modified` datetime NOT NULL COMMENT '更新时间',
14  PRIMARY KEY (`id`) USING BTREE,
15  KEY `idx_order_id` (`order_id`) USING BTREE
16 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COMMENT='订单售后条目表';

```

after_sale_refund (售后退款表)

```

1 CREATE TABLE `after_sale_refund` (
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键id',
3   `after_sale_id` varchar(64) NOT NULL COMMENT '售后单号',
4   `order_id` varchar(64) NOT NULL COMMENT '订单号',
5   `after_sale_batch_no` varchar(64) DEFAULT NULL COMMENT '售后批次编号',
6   `account_type` tinyint(4) NOT NULL COMMENT '账户类型',
7   `pay_type` tinyint(4) NOT NULL COMMENT '支付类型',
8   `refund_status` tinyint(4) NOT NULL COMMENT '退款状态',
9   `refund_amount` int(11) NOT NULL COMMENT '退款金额',
10  `refund_pay_time` datetime DEFAULT NULL COMMENT '退款支付时间',
11  `out_trade_no` varchar(64) DEFAULT NULL COMMENT '交易单号',
12  `remark` varchar(1024) DEFAULT NULL COMMENT '备注',
13  `gmt_create` datetime NOT NULL COMMENT '创建时间',
14  `gmt_modified` datetime NOT NULL COMMENT '更新时间',
15  PRIMARY KEY (`id`) USING BTREE
16 ) ENGINE=InnoDB DEFAULT CHARSET=utf8 COMMENT='售后支付表';

```

after_sale_log (售后单变更表)

```
1 CREATE TABLE `after_sale_log` (  
2   `id` bigint(20) NOT NULL AUTO_INCREMENT COMMENT '主键id',  
3   `after_sale_id` varchar(50) NOT NULL COMMENT '售后单号',  
4   `pre_status` tinyint(4) NOT NULL COMMENT '前一个状态',  
5   `current_status` tinyint(4) NOT NULL COMMENT '当前状态',  
6   `remark` varchar(1024) NOT NULL COMMENT '备注',  
7   `gmt_create` datetime NOT NULL COMMENT '创建时间',  
8   `gmt_modified` datetime NOT NULL COMMENT '更新时间',  
9   PRIMARY KEY (`id`) USING BTREE  
10 ) ENGINE=InnoDB AUTO_INCREMENT=40 DEFAULT CHARSET=utf8 COMMENT='售后单变更表';
```

正向订单相关枚举

订单类型 (orderType)

```
1 public enum OrderTypeEnum {  
2     NORMAL(1, "一般订单"),  
3     UNKNOWN(127, "其他");  
4 }
```

业务线 (businessIdentifier)

```
1 public enum BusinessIdentifierEnum {  
2     SELF_MALL(1, "自营商城");  
3 }
```

订单状态 (orderStatus)

```
1 public enum OrderStatusEnum {
2
3     NULL(0, "未知"),
4     CREATED(10, "已创建"),
5     PAID(20, "已支付"),
6     FULFILL(30, "已履约"),
7     OUT_STOCK(40, "出库"),
8     DELIVERY(50, "配送中"),
9     SIGNED(60, "已签收"),
10    CANCELED(70, "已取消"),
11    REFUSED(100, "已拒收"),
12    INVALID(127, "无效订单");
13 }
```

支付类型 (payType)

```
1 public enum PayTypeEnum {
2
3     WECHAT_PAY(10, "微信支付"),
4     ALI_PAY(20, "支付宝支付");
5 }
```

支付状态 (payStatus)

```
1 public enum PayStatusEnum {
2
3     UNPAID(10, "未支付"),
4     PAID(20, "已支付");
5 }
```

费用类型 (amountType)


```
1 public enum AmountTypeEnum {
2
3     ORIGIN_PAY_AMOUNT(10, "订单支付原价"),
4     COUPON_DISCOUNT_AMOUNT(20, "优惠券抵扣金额"),
5     SHIPPING_AMOUNT(30, "运费"),
6     BOX_AMOUNT(40, "包装费"),
7     REAL_PAY_AMOUNT(50, "实付金额"),
8     OTHER_AMOUNT(127, "其他费用");
9 }
```

账户类型 (accountType)

```
1 public enum AccountTypeEnum {
2
3     THIRD(1, "第三方"),
4     OTHER(127, "其他");
5 }
```

配送方式 (deliveryType)

```
1 public enum DeliveryTypeEnum {
2
3     NULL(0, "无配送方式"),
4     SELF(1, "自主配送");
5 }
```

订单取消类型 (orderCancelType)

Java | [复制代码](#)

```
1 public enum OrderCancelTypeEnum {
2
3     USER_CANCELED(0, "用户手动取消"),
4     TIMEOUT_CANCELED(1, "超时未支付自动取消"),
5     CUSTOMER_CANCELED(2, "用户授权客服取消");
6 }
```

平台 (platForm)

Java | [复制代码](#)

```
1 public enum PlatformEnum {
2
3     IOS(10, "苹果APP"),
4     ANDROID(20, "安卓APP"),
5     WECHAT_APP(30, "微信小程序"),
6     WECHAT_H5(40, "微信H5"),
7     OTHER(127, "其他");
8 }
```

商品类型 (productType)

Java | [复制代码](#)

```
1 public enum ProductTypeEnum {
2
3     NORMAL_PRODUCT(1, "普通商品"),
4     ADVANCE_SALE(2, "预售商品");
5 }
```

订单查询类型 (QueryDataType)

```
1 public enum QueryDataTypeEnums {
2     /**订单基础**/
3     BASE,
4     /**订单条目**/
5     ORDER_ITEM,
6     /**订单费用明细**/
7     ORDER_AMOUNT_DETAIL,
8     /**订单支付**/
9     PAYMENT,
10    /**订单配送信息**/
11    DELIVERY,
12    /**售后**/
13    AFTER_SALE,
14    /**售后明细**/
15    AFTER_SALE_DETAIL,
16    /**售后交易**/
17    AFTER_SALE_PAY,
18    /**渠道**/
19    CHANNEL,
20    /**费用**/
21    AMOUNT,
22    /**操作日志**/
23    OPERATE_LOG,
24    /**订单快照**/
25    SNAPSHOT
26    ;
27 }
```

逆向售后相关枚举

售后申请来源枚举

```
1 public enum AfterSaleApplySourceEnum {
2     USER_APPLY(10, "用户申请退款"),
3     SYSTEM(20, "系统自动退款"),
4     CUSTOM_APPLY(30, "客服申请退款"),
5     USER_RETURN_GOODS(40, "用户申请退货");
6 }
```

售后申请原因枚举

去掉 平台仲裁取消

Java

[复制代码](#)

```
1 public enum AfterSaleReasonEnum {
2     ITEM_NUM(10, "商品数量原因"),
3     ITEM_QUALITY(20, "商品质量原因"),
4     ITEM_PACKAGE(30, "商品包装原因"),
5     LOGISTICS(40, "物流原因"),
6     DELIVERY(50, "快递员原因"),
7     USER(60, "用户自己原因"),
8     ITEM_PRICE(70, "商品价格原因"),
9     CANCEL(80, "取消订单"),
10    FORCED_CANCELLATION(90, "平台强制取消订单"),
11    DISHONOR(100, "拒收"),
12    OTHER(127, "其他");
13 }
```

售后明细类型

商品类型:0-普通商品,1-优惠券,2-运费

Java

[复制代码](#)

```
1 public enum AfterSaleDetailEnum {
2     ITEM(0, "普通商品"),
3     COUPON(1, "优惠券"),
4     FREIGHT(2, "运费");
5 }
```

售后支付状态枚举 (AfterSalePaymentStatus)

1、refundPaymentStatus->AfterSalePaymentStatus

```

1  public enum AfterSalePaymentStatusEnum {
2      APPLYING(10, "申请中"),
3      REFUNDING(20, "退款中"),
4      REFUNDED(30, "退款成功"),
5      FAILED(40, "退款失败");
6  }

```

售后单状态枚举 (AfterSaleStatus)

- 1、refundStatus->AfterSaleStatus
- 2、VERIFY->REVIEW
- 3、去掉系统审核通过和拒绝(SYSTEM_VERIFY_SUCCESS/FAILED)

```

1  public enum AfterSaleStatusEnum {
2      COMMITED(10, "提交申请"),
3      REVIEW_PASS(20, "审核通过"),
4      REVIEW_REJECTED(30, "审核拒绝"),
5      REFUNDING(40, "退款中"),
6      REFUNDED(50, "退款成功"),
7      FAILED(60, "退款失败"),
8      CLOSED(70, "已关闭"),
9      REOPEN(100, "重新提交申请"),
10     REVOKE(127, "撤销成功");
11 }

```

售后类型 (AfterSaleType)

- 1、refundType->AfterSaleType
- 2、第一版只做整单退款的情况，其他的都去掉

```
1 public enum AfterSaleTypeEnum {
2     ALL_REFUND(1, "售后全额退款"),
3     TIMEOUT_NO_PAY(2, "超时未支付"),
4     USER_CANCEL(3, "用户自主取消"),
5     CUSTOMER_CANCEL(4, "授权客服取消"),
6     AFTER_SALE_RETURN_ALL_GOODS(5, "先退货后退款：全部退货"),
7     AFTER_SALE_REFUND(6, "先退货后退款：已退款"),
8     REFUND(7, "直接退款"),
9     LACK_REFUND(8, "缺品退款"),
10    REFUNDED(10, "已退款");
11 }
```

二、接口设计

1、正向订单业务

1.1、生成订单号接口

接口定义：

```
1 /**
2  * 生成订单号接口
3  *
4  * @param genOrderIdRequest 生成订单号入参
5  * @return 订单号
6  */
7 JsonResult<GenOrderIdDTO> genOrderId(GenOrderIdRequest genOrderIdRequest);
```

请求入参：

(示例)

JSON | 复制代码

```
1  {
2      "businessIdentifier":1,
3      "userId":100
4  }
```

返回结果：

(示例)

主要返回新生成的订单ID

JSON | 复制代码

```
1  {
2      "success": true,
3      "data": {
4          "orderId": "1021120232956689100"
5      },
6      "errorCode": null,
7      "errorMessage": null
8  }
```

使用场景：

调用方提交订单前需要先调用该接口获取到一个正式的订单号，然后作为下单的接口的入参。

具体实现：

- 19位，2位是业务类型，比如10开头是正向，20开头是逆向，然后中间6位是日期，然后中间8位是序列号，最后3位是用户ID后三位
 - 8位序列号基于数据自增主键生成
 - 用户ID不足3位前面补0

```
1  /** 订单号类型枚举 */
2  public enum OrderNoTypeEnum {
3      SALE_ORDER("10", "正向订单号"),
4      AFTER_SALE("20", "售后单号");
5  }
```

1.2、提交订单接口

接口定义：

```
1  /**
2   * 提交订单接口
3   * @param createOrderRequest 提交订单请求入参
4   * @return 订单号
5   */
6  JsonResult<CreateOrderDTO> createOrder(CreateOrderRequest
    createOrderRequest);
```

请求入参：

(示例)


```
1  {
2    "orderId":1021120232956689100,
3    "businessIdentifier":1,
4    "openid":null,
5    "userId":100,
6    "orderType":1,
7    "sellerId":101,
8    "userRemark":"test reamark",
9    "couponId":"1001004",
10   "deliveryType":1,
11   "province":"110000",
12   "city":"110100",
13   "area":"110105",
14   "street":"110101007",
15   "detailAddress":"北京路10号",
16   "lon":100.10000,
17   "lat":1010.201010,
18   "receiverName":"张三",
19   "receiverPhone":"13434545545",
20   "userAddressId":"1010",
21   "addressCode":"1010100",
22   "regionId":"10002020",
23   "shippingAreaId":"101010212",
24   "clientIp":"34.53.12.34",
25   "deviceId":"45sf2354adfw245",
26   "orderItemRequestList":[
27     {
28       "productType":1,
29       "saleQuantity":10,
30       "skuCode":"10101010"
31     },
32     {
33       "productType":2,
34       "saleQuantity":1,
35       "skuCode":"10101011"
36     }
37   ],
38   "orderAmountRequestList":[
39     {
40       "amountType":10,
41       "amount":10100
42     },
43     {
44       "amountType":20,
45       "amount":500
46     },
47     {
48       "amountType":30,
```

```

49         "amount":0
50     },
51     {
52         "amountType":50,
53         "amount":9600
54     }
55 ],
56 "paymentRequestList":[
57     {
58         "payType":10,
59         "accountType":1
60     }
61 ]
62 }

```

返回结果：

(示例)

```

1  {
2      "success": true,
3      "data": {
4          "orderId": "1021120232958737100"
5      },
6      "errorCode": null,
7      "errorMessage": null
8  }

```

JSON

[复制代码](#)

使用场景：

用户选购好商品，确认提交订单时调用此接口完成生单。

具体实现：

- 入参校验，不通过就抛异常
- 调用风控服务进行风控检查，不通过就抛异常
- 调用商品服务，查询订单条目商品详情
- 调用营销服务，计算订单价格
 - 运费计算

- 1 通过营销中心的运费模板表来计算出订单所需运费。

○ 优惠券折扣分摊

```

1  # 优惠券分摊算法
2
3  {
4  //实付金额
5  {"realPay":103元
6   "type":1
7  }
8  //原支付金额
9  {"amount": 108元
10 "type":2
11 }
12 //运费
13 {"deliveryAmount":3
14 "type":4
15 }
16 //优惠券优惠金额
17 {"couponamount":5
18
19 //币 + 红包 + 券包
20
21 "type":7}
22 //商品金额明细
23 "itemAmount":{
24   {sku:'苹果',price:'3',quantity:2,realPay:3.5},
25   {sku:'李子',price:'3',quantity:2,realPay:3.5}
26 }
27 //原价 = sku金额 * 数量 + 运费
28 //实付 = sku金额 * 数量 + 运费 - 总优惠金额
29       = sku的实付 + 运费
30
31 苹果的优惠分摊金额 =  $5 * (3 * 2) / (3 * 2 + 3 * 2) = 2.5$ 
32 苹果的实付金额 =  $3 * 2 - 2.5 = 3.5$ 
33
34
35 如果计算出的优惠券分摊到一条 item 上存在小数时，则向上取整，
36 然后最后一条 item 分摊的金额就用优惠金额减掉前面所有优惠的item分摊的总额。

```

- 订单验价（主要验证服务端计算出来的订单支付金额与前端传递过来的支付金额是否一致）

```

1  # 订单验价逻辑
2
3  前端会给一个商品售卖价
4
5  查询商品服务的时候-商品服务会给一个价格（替换前端的）
6
7  算价/营销中心【sku会员价】也会讲价格算出来->比较算出来的总金额 和前端给的总金额对比 不一样就说明下单那一刻金额变化

```

- 调用营销服务，锁定优惠券
- 调用库存服务，锁定商品库存
- 保存订单数据到数据库
 - 如果有多种商品类型，需要拆单

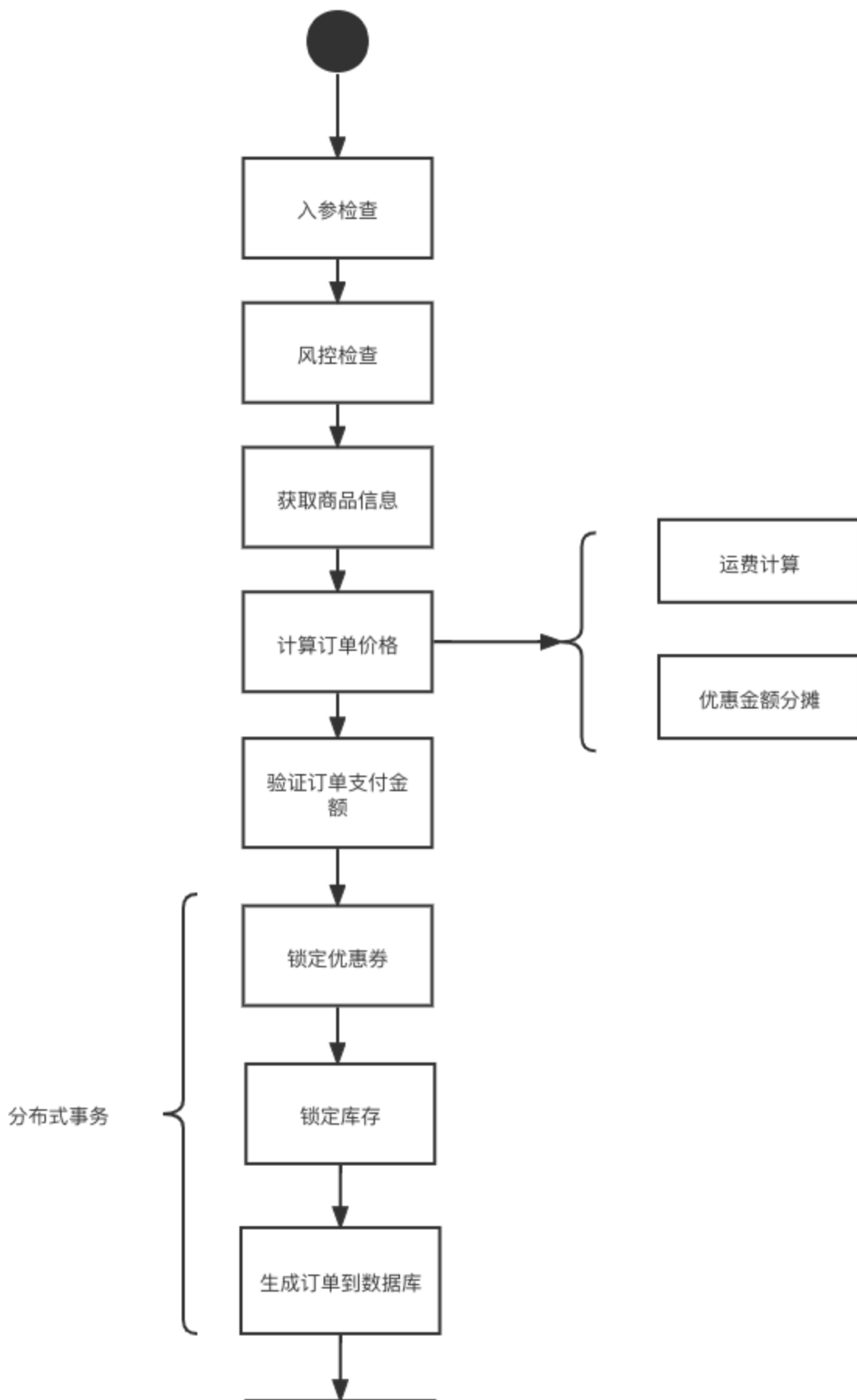
```

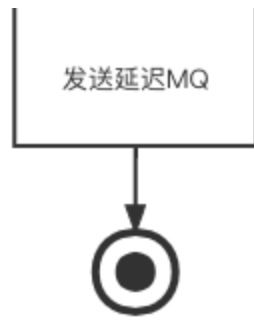
1  # 拆单的逻辑
2
3  sku:苹果 sku:预售品
4  sku:苹果订单先发货
5
6  order,2个sku, 其他信息基本都是一样除了钱
7
8  1.假设我new order , 先去获取一个orderId
9  2.paarentOrderId填父单的
10 3.地址风控用户都一样
11 4.钱就是要换一下
12 5.拆单是在算价之前（营销去算价就行）
13 6.构造出内存中三个订单，父单状态作废，子单状态10，orderstatuslog（父单10-255 子单10-20）
14 6.插库 插入进3个单

```

- 订单数据相关表
 - 订单信息表（order_info）
 - 订单条目表（order_item）
 - 订单配送信息表（order_delivery_detail）
 - 订单支付明细表（order_payment_detail）
 - 订单费用表（order_amount）
 - 订单费用明细表（order_amount_detail）
 - 订单状态变更日志表（order_operate_log）
 - 订单快照信息表（order_snapshot）

- 发送延迟mq消息，用于超时未支付取消订单





技术要点：

- 扣减库存+锁定优惠券+创建订单 分布式事务解决方案
使用seata的AT模式来解决。

- 超时取消订单处理方案
生成订单之后发送一个延迟MQ来出来，MQ的消费方来消费消息，然后调用取消订单的接口。

- 订单信息组建
采用builder构建者设计模式来实现。

```

1  NewOrderBuilder newOrderBuilder = new NewOrderBuilder(createOrderRequest,
2      \
3      productSkuList,
4      calculateOrderAmountDTO,
5      orderProperties);
6  FullOrderData fullOrderData = newOrderBuilder
7      // 订单信息
8      .buildOrder()
9      // 订单条目
10     .buildOrderItems()
11     // 订单配送信息
12     .buildOrderDeliveryDetail()
13     // 订单支付明细
14     .buildOrderPaymentDetail()
15     // 订单费用信息
16     .buildOrderAmount()
17     // 订单费用明细
18     .buildOrderAmountDetail()
19     // 订单状态变更日志
20     .buildOperateLog()
21     // 订单快照信息
22     .buildOrderSnapshot()
23     .build();

```

1.3、预支付订单接口

接口定义：

```

1  /**
2   * 预支付订单接口
3   * @param prePayOrderRequest 预支付订单请求入参
4   * @return
5   */
6  JsonResult<PrePayOrderDTO> prePayOrder(PrePayOrderRequest
    prePayOrderRequest);

```

请求入参：

JSON | 复制代码

```
1  {
2      "userId":"100",
3      "businessIdentifier":1,
4      "payType":10,
5      "orderId":"1021120232956929100",
6      "payAmount":"9600",
7      "callbackUrl":"http://pay.ruyuan.com/order/callbackUrl",
8      "callbackFailUrl":"http://pay.ruyuan.com/order/callbackFailUrl",
9      "openid":null,
10     "subject":"新鲜水果",
11     "itemInfo":"test"
12 }
```

返回结果：

JSON | 复制代码

```
1  {
2      "success": true,
3      "data": {
4          "orderId": "1021120232956929100",
5          "outTradeNo": "4236081648418886561",
6          "payType": 10,
7          "payData": {
8              "payAmount": 9600,
```

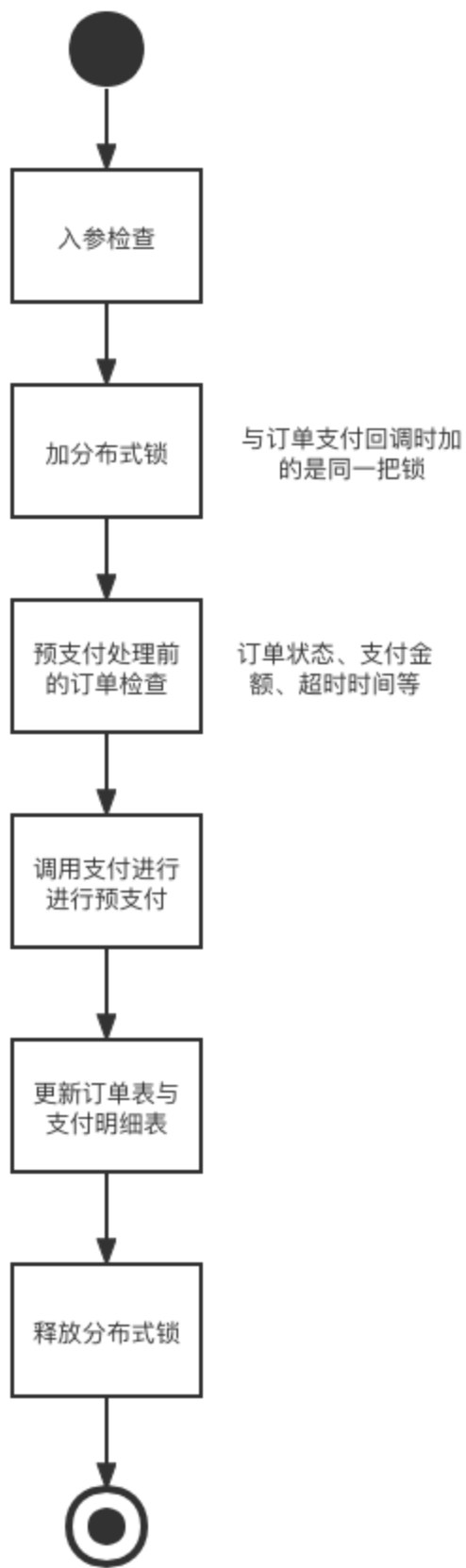
使用场景：

用户完成订单提交之后需要对订单进行支付，点击“去支付”按钮后，系统调用此接口完成订单与支付，成功之后前端引导用户跳转到支付界面。

具体实现：

- 查询订单信息
- 校验订单信息（支付方式，订单信息是否存在、状态是否正确、是否超过支付取消时间）
- 加支付分布式锁
- 组装请求参数

- 调用支付系统进行预支付
- 更新（是否拆单）订单表与支付信息表（更新这两个表的支付类型与支付时间）
- 如果存在拆单，则还需要更新子订单



技术要点：

- 多线程并发安全解决方案

用户可能触发多次订单支付，也就是可能会调用多次预支付接口。预支付订单接口会更改订单与支付记录，支付回调时也会判断订单支付状态等，所以预支付与支付回调两个接口需要基于分布式锁来保证数据并发安全性。

采用redisson框架来实现分布式锁。

1.4、支付回调接口

接口定义：

Java | 复制代码

```
1  /**
2   * 支付回调接口
3   * @param payCallbackRequest 支付系统回调入参
4   * @return
5   */
6  JsonResult<Boolean> payCallback(PayCallbackRequest payCallbackRequest);
```

请求入参：

JSON | 复制代码

```
1  {
2      "orderId": "1021120232956929100",
3      "payAccount": "test.qq.com",
4      "payAmount": "9600",
5      "outTradeNo": "4236081648418886561",
6      "payType": "10",
7      "merchantId": "10101010",
8      "payChannel": "wechat_pay",
9      "appid": "wx207d34495e588e1c"
10 }
```

返回结果：

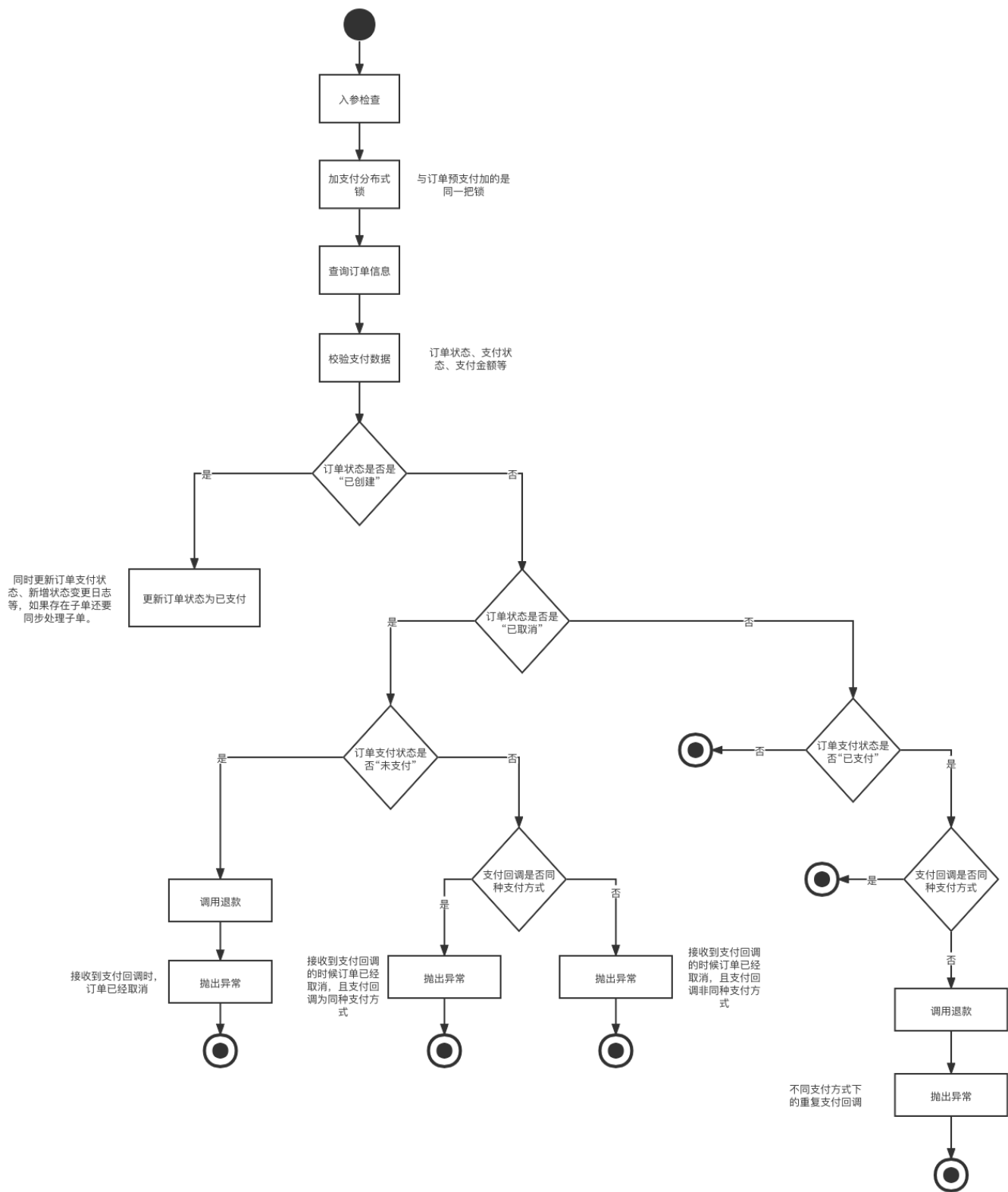
无 (void)

使用场景：

此接口是支付系统完成了用户支付操作之后自动来回调的，不是前端应用来发起调用的。

具体实现：

- 回调入参检查
- 添加分布式锁，要与预支付接口使用同一把锁
- 订单状态与支付信息判断
- 异常场景判断
 - 正常情况下会修改订单状态与支付状态为已支付，并记录操作日志。
 - 异常情况下则需要分类进行判断，做各种处理。



技术要点：

- 由于用户可能发起多次预支付调用，所以预支付与支付回调两个接口需要使用同一把分布式锁来保证并发数据处理安全。

1.5、触发订单履约流程接口

所属系统：订单系统(ruyuan-eshop-order-service)

调用时机

订单系统监听到“订单已支付”消息时触发

实现方法

OrderFulFillService.triggerOrderFulFill()

接口定义

Java [复制代码](#)

```
1  /**
2   * 触发订单进行履约流程
3   * @param orderId
4   * @return
5   */
6  public void triggerOrderFulFill(String orderId);
```

请求入参

JSON [复制代码](#)

```
1  {
2   // 订单号
3   "orderId":"1021231231"
4  }
```

返回结果

JSON

 复制代码

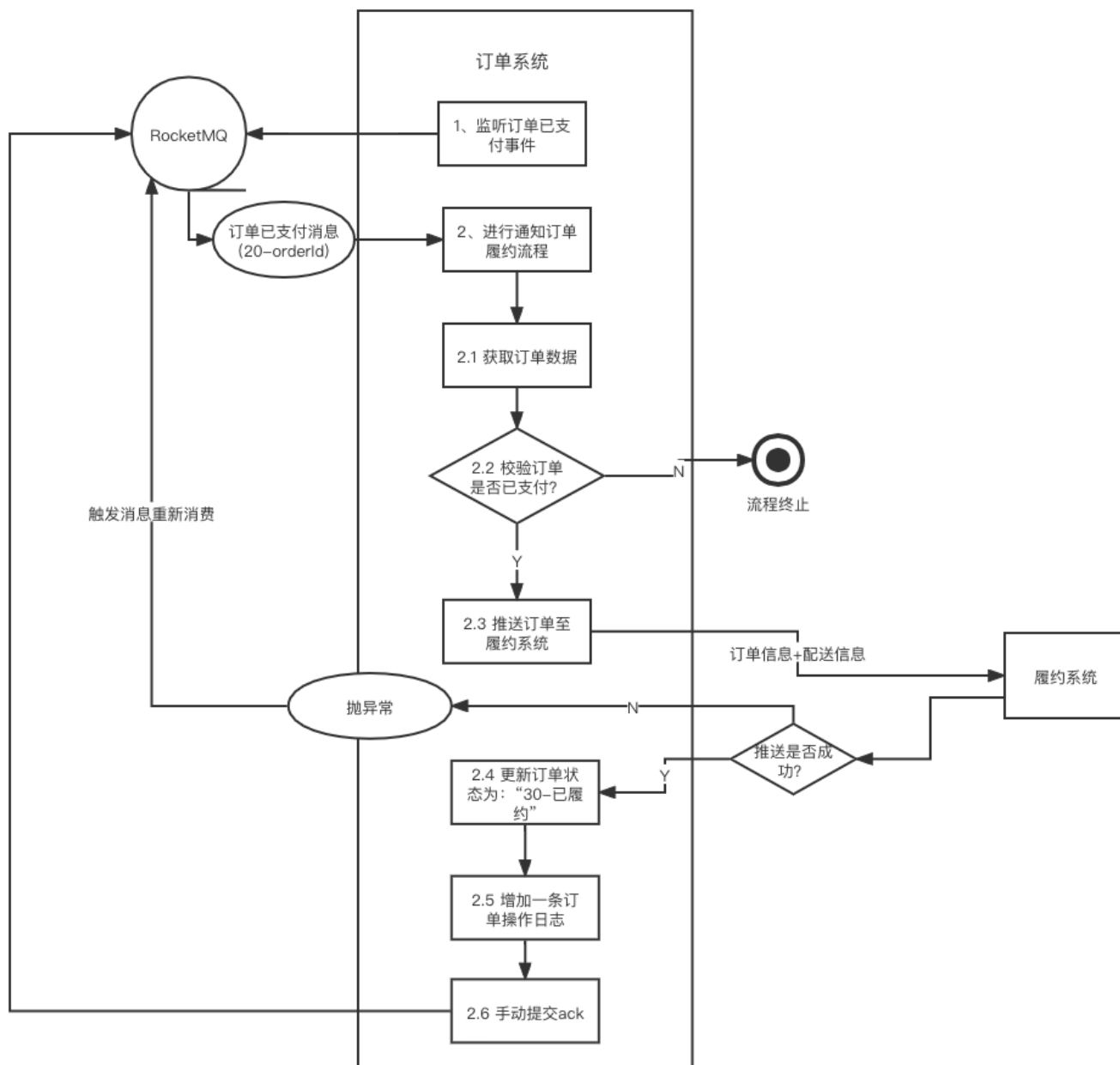
```
1 void
```

业务规则

支付成功的订单需要推送给履约系统进行履约

业务流程

1. 订单系统监听RocketMQ的订单已支付消息
2. 订单系统收到订单已支付消息，根据消息中的orderId查询出一笔订单
3. 校验订单是否已支付，如果不是，直接终止流程
4. 构造履约请求，调用履约系统的履约接口
5. 如果调用失败，抛异常，触发RocketMQ的消息重复消费功能
6. 如果调用成功，更新订单状态为：“已履约”，并插入一条订单变更记录
7. 手动提交ack



技术要点

1. 订单系统调用履约系统接口失败时，需要抛异常，触发RocketMQ的消息重复消费功能，重新执行订单履约流程
2. 消息消费幂等，通过分布式锁+状态前置校验实现
3. 分布式锁加锁应该放在本地事务外面，而不能将加锁包裹本地事务里面
4. 需要考虑调用履约系统接口成功，但是本地事务执行失败单可能，即需要保证调用履约系统接口和本地事物执行的最终一致性，目前第一版暂未实现，二期会使用RocketMQ的事务消息实现

1.6、接受订单履约接口

所属系统：履约系统(ruyuan-eshop-fulfill-service)

接口说明

一个mock接口，主要用来模拟业务系统间的交互

调用时机

订单系统触发履约流程时会调用该接口，将订单履约信息推送给履约系统

实现方法

FulfillApi.receiveOrderFulFill()

接口定义

Java | [复制代码](#)

```
1  /**
2   * 接收订单履约
3   * @return
4   */
5  public JsonResult<Boolean> receiveOrderFulFill(ReceiveFulFillDTO request);
```

请求入参

```
1
2  /**
3   * <p>
4   *   接受订单履约请求
5   * </p>
6   *
7   * @author zhonghuashishan
8   */
9  @Data
10 @Builder
11 public class ReceiveFulFillRequest implements Serializable {
12
13     /**
14     * 订单号
15     */
16     private String orderId;
17
18     /**
19     * 商家id
20     */
21     private String sellerId;
22
23     /**
24     * 用户id
25     */
26     private String userId;
27
28     /**
29     * 配送类型，默认是自配送
30     */
31     private Integer deliveryType;
32
33     /**
34     * 收货人姓名
35     */
36     private String receiverName;
37
38     /**
39     * 收货人电话
40     */
41     private String receiverPhone;
42
43     /**
44     * 省
45     */
46     private String receiverProvince;
47
48     /**
```

```
49     * 市
50     */
51     private String receiverCity;
52
53     /**
54     * 区
55     */
56     private String receiverArea;
57
58     /**
59     * 街道地址
60     */
61     private String receiverStreetAddress;
62
63     /**
64     * 详细地址
65     */
66     private String receiverDetailAddress;
67
68     /**
69     * 经度 六位小数点
70     */
71     private BigDecimal receiverLon;
72
73     /**
74     * 纬度 六位小数点
75     */
76     private BigDecimal receiverLat;
77
78     /**
79     * 商家备注
80     */
81     private String shopRemark;
82
83     /**
84     * 用户备注
85     */
86     private String userRemark;
87
88     /**
89     * 支付方式
90     */
91     private Integer payType;
92
93     /**
94     * 付款总金额
95     */
96     private Integer payAmount;
97
98     /**
```

```

99         * 交易总金额
100        */
101        private Integer totalAmount;
102
103        /**
104         * 运费
105        */
106        private Integer deliveryAmount;
107
108        /**
109         * 订单商品明细
110        */
111        private List<ReceiveOrderItemRequest> receiveOrderItems;
112
113        @Tolerate
114        public ReceiveFulFillRequest() {
115        }
116    }
117
118    /**
119     * <p>
120     * 履约订单商品明细请求
121     * </p>
122     *
123     * @author zhonghuashishan
124     */
125    @Data
126    @Builder
127    public class ReceiveOrderItemRequest implements Serializable {
128
129        /**
130         * 商品id
131        */
132        private String skuCode;
133
134        /**
135         * 商品名称
136        */
137        private String productName;
138
139        /**
140         * 销售单价
141        */
142        private Integer salePrice;
143
144        /**
145         * 销售数量
146        */
147        private Integer saleQuantity;
148    }

```

```

149     /**
150      * 商品单位
151      */
152     private String productUnit;
153
154     /**
155      * 付款金额
156      */
157     private Integer payAmount;
158
159     /**
160      * 当前商品支付原总价
161      */
162     private Integer originAmount;
163
164     @Tolerate
165     public ReceiveOrderItemRequest() {
166
167     }
168
169 }
170

```

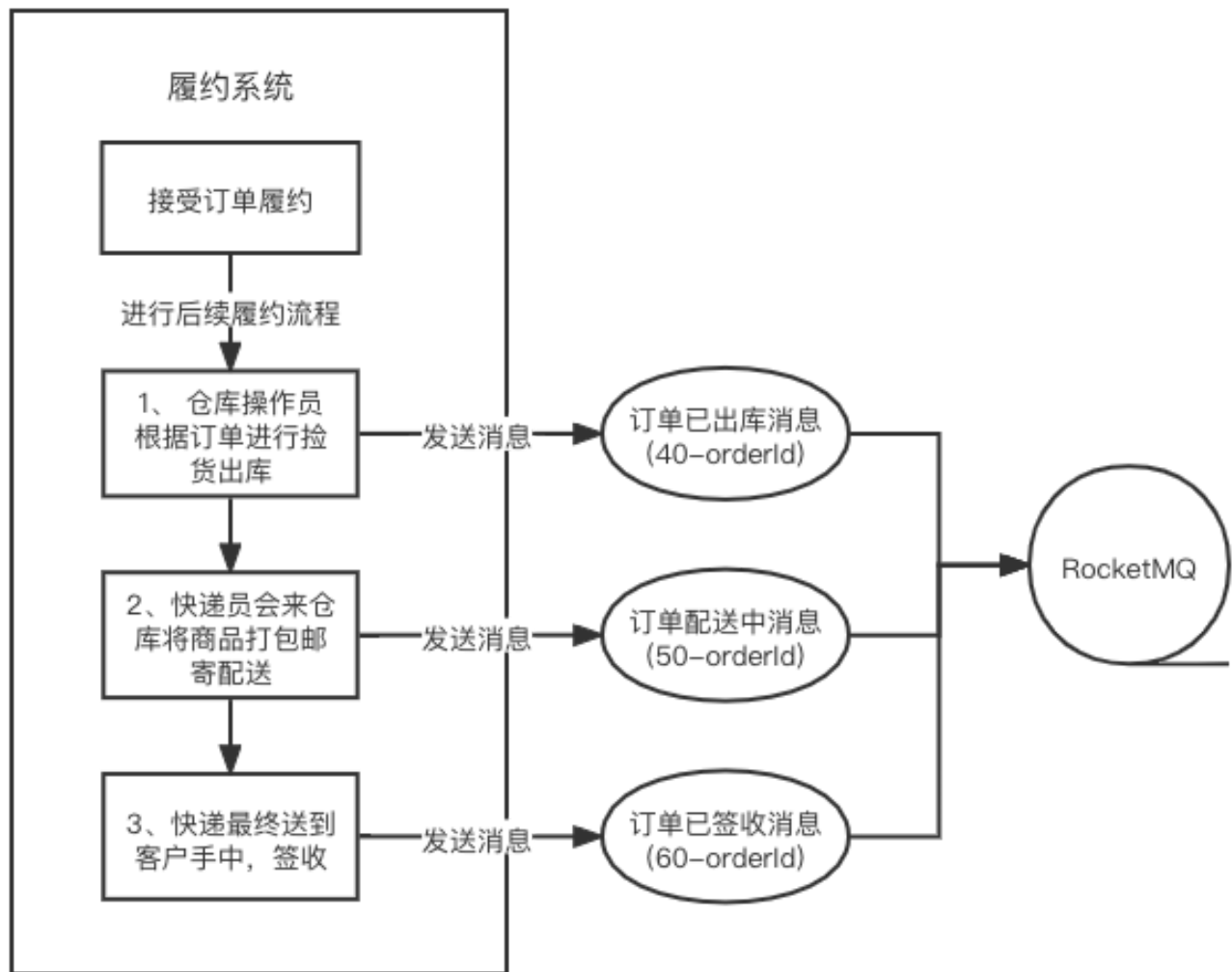
返回结果

1 **true/false** 成功/失败

JSON | [复制代码](#)

业务流程

履约服务需要提供该接口进行后续的订单履约操作，在这里，履约系统将扮演“仓储+物流”的角色，进行后续的商品捡货出库，商品打包邮寄配送，以及包裹签收的流程，具体如下所示



1.7、触发订单物流配送结果事件接口

所属系统：履约系统(ryuan-eshop-fulfill-service)

接口说明

一个工具类接口，用于模拟触发履约后的流程，触发：“订单已出库事件”，“订单已配送事件”，“订单已签收事件”

实现方法

FulfillApi.triggerOrderWmsShipEvent()

接口定义

Java | [复制代码](#)

```
1  /**
2   * 触发订单物流配送结果事件接口
3   * 一个工具类接口，用于模拟触发"订单已出库事件"，"订单已配送事件"，"订单已签收事件"
4   *
5   * @param orderId          订单号
6   * @param orderStatusChange 订单状态
7   * @param wmsEvent         物流结果事件
8   * @return
9   */
10 JsonResult<Boolean> triggerOrderWmsShipEvent(String orderId
11         , OrderStatusChangeEnum orderStatusChange, BaseWmsShipEvent
12         wmsEvent);
```

请求入参

```

1  /**
2   * 订单状态变化枚举
3   */
4  @Getter
5  public enum OrderStatusChangeEnum {
6      /**
7       * 订单已履约
8       */
9
10     ORDER_FULFILLED(OrderStatusEnum.PAID,OrderStatusEnum.FULFILL,OrderOperate
TypeEnum.PUSH_ORDER_FULFILL),
11     /**
12      * 订单已出库
13      */
14
15     ORDER_OUT_STOCKED(OrderStatusEnum.FULFILL,OrderStatusEnum.OUT_STOCK,Order
OperateTypeEnum.ORDER_OUT_STOCK),
16     /**
17      * 订单已配送
18      */
19
20     ORDER_DELIVERED(OrderStatusEnum.OUT_STOCK,OrderStatusEnum.DELIVERY,OrderO
perateTypeEnum.ORDER_DELIVERED),
21     /**
22      * 订单已签收
23      */
24
25     ORDER_SIGNED(OrderStatusEnum.DELIVERY,OrderStatusEnum.SIGNED,OrderOperate
TypeEnum.ORDER_SIGNED),
26     ;
27 }
28
29 orderId: 订单号
30
31 /**
32  * 物流配送结果事件基类

```

返回结果

```

1  true/false : 成功/失败

```


业务流程

根据传入的参数，构造“订单已出库事件”，“订单已配送事件”，“订单已签收事件”并发送到对应的topic上去

1.8、通知订单物流配送结果接口

所属系统：订单系统(ryuan-eshop-order-service)

调用时机

监听到订单物流配送结果事件（“订单已出库事件”，“订单已配送事件”，“订单已签收事件”）时触发，

实现方法

OrderFulFillService.informOrderWmsShipResult()

接口定义

```
1  /**
2   * 通知订单物流配送结果接口
3   * @return
4   */
5  void informOrderWmsShipResult(WmsShipDTO wmsShipDTO);
```

Java | [复制代码](#)

请求参数

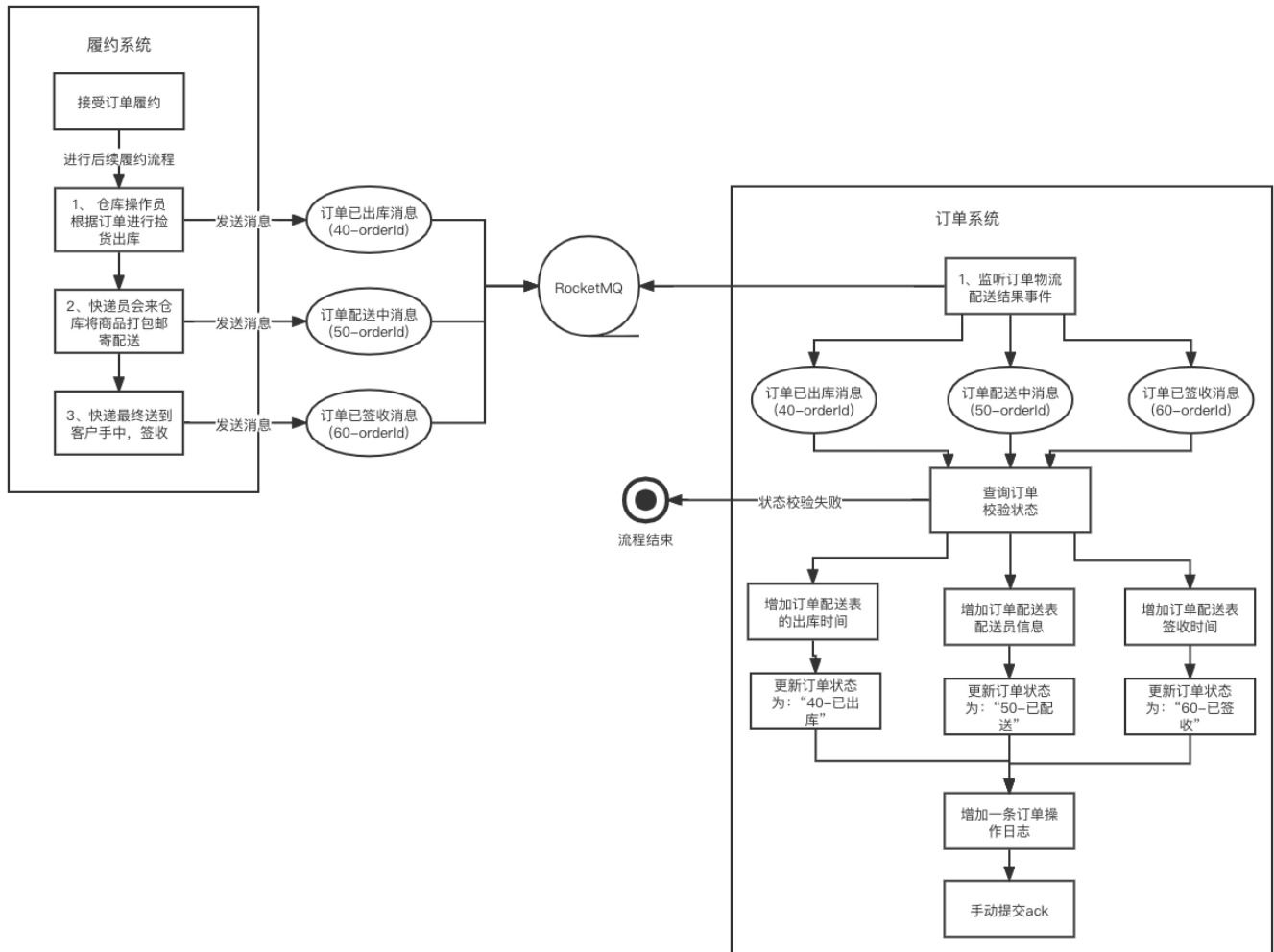
```
1  /**
2   * 物流配送结果
3   *
4   * @author zhonghuashishan
5   */
6  @Data
7  public class WmsShipDTO {
8
9      /**
10       * 订单编号
11       */
12     private String orderId;
13     /**
14      * 订单状态变更
15      */
16     private OrderStatusChangeEnum statusChange;
17     /**
18      * 出库时间
19      */
20     private Date outStockTime;
21     /**
22      * 签收时间
23      */
24     private Date signedTime;
25     /**
26      * 配送员code
27      */
28     private String delivererNo;
29     /**
30      * 配送员姓名
31      */
32     private String delivererName;
33     /**
34      * 配送员手机号
35      */
36     private String delivererPhone;
37 }
```

返回结果

```
1  void
```

业务流程

1. 订单系统监听到“订单已出库消息”
 - a. 校验订单状态，如果订单状态不是“已履约”，流程直接结束
 - b. 订单配送表增加出库时间
 - c. 更新订单状态为“已出库”，并增加一条操作日志
2. 订单系统监听到“订单已配送消息”
 - a. 校验订单状态，如果订单状态不是“已出库”，流程直接结束
 - b. 订单配送表增加配送员信息（编号，手机，姓名）
 - c. 更新订单状态为“已配送”，并增加一条操作日志
3. 订单系统监听到“订单已签收消息”
 - a. 校验订单状态，如果订单状态不是“已签收”，流程直接结束
 - b. 订单配送表增加签收时间
 - c. 更新订单状态为“签收”，并增加一条操作日志



技术要点

1. 本地业务逻辑需要加本地数据库事务，当本地事务执行失败时需要抛异常，触发RocketMQ的消息重复消费功能，重新执行订单履约流程
2. 消息消费幂等，通过分布式锁+状态前置校验实现
3. 分布式锁加锁应该放在本地事务外面，而不能将加锁包裹本地事务里面

实现亮点：

1. 通过模板方法模式规范处理流程

```
1  public void execute(WmsShipDTO wmsShipDTO) throws OrderBizException {
2
3      //1、查询订单
4      OrderInfoDO order =
orderInfoDAO.getByOrderId(wmsShipDTO.getOrderId());
5      if(null == order) {
6          return;
7      }
8
9      //2、校验订单状态
10     if(!checkOrderStatus(order)) {
11         return;
12     }
13
14     //3、执行具体的业务逻辑
15     doExecute(wmsShipDTO,order);
16
17     //4、更新订单状态
18     changeOrderStatus(order,wmsShipDTO);
19
20     //5、增加操作日志
21     saveOrderOperateLog(order,wmsShipDTO);
22 }
```

1.9、删除订单

所属系统：订单系统(ruyuan-eshop-order-service)

实现方法

OrderApi.removeOrders()

接口定义

```
1  /**
2   * 移除订单
3   * @param removeOrderRequest 移除订单请求入参
4   * @return
5   */
6  JsonResult<RemoveOrderDTO> removeOrders(RemoveOrderRequest
    removeOrderRequest);
```

请求参数

```
1  /**
2   *
3   * 移除订单的请求
4   * @author zhonghuashishan
5   * @version 1.0
6   */
7  @Data
8  public class RemoveOrderRequest extends AbstractObject implements
    Serializable {
9      /**
10     * 要移除的订单ids
11     */
12     private Set<String> orderIds;
13 }
```

返回结果

```
1  /**
2   * 移除订单的响应
3   * @author zhonghuashishan
4   * @version 1.0
5   */
6  @Data
7  public class RemoveOrderDTO extends AbstractObject implements Serializable
8  {
9      /**
10     * 响应结果
11     */
12     private boolean result;
13 }
```

业务规则

删除是软删除，删除后用户不可查询，只有签收之后才可以删除

具体实现

1. 对传入的orderIds参数进行长度校验，长度超过200，直接抛异常
2. 根据orderIds查询出来一批订单
3. 接着对每笔订单进行校验，校验是否是签收后状态（60，70，200），如果不是，直接抛异常
4. 对查询出来每次订单进行删除操作，将其deleteStatus字段设置为“1-已删除”
5. 构造返参，返回给调用方

1.10、修改配送地址

所属系统：订单系统(ryuan-eshop-order-service)

实现方法

OrderApi.adjustDeliveryAddress()

接口定义

Java | [复制代码](#)

```
1  /**
2   * 修改地址
3   * @param adjustDeliveryAddressRequest 修改地址请求入参
4   * @return
5   */
6  JsonResult<AdjustDeliveryAddressDTO>
    adjustDeliveryAddress(AdjustDeliveryAddressRequest
        adjustDeliveryAddressRequest);
7
```

请求参数


```
1  /**
2   * 调整订单配送地址
3   * @author zhonghuashishan
4   * @version 1.0
5   */
6  @Data
7  public class AdjustDeliveryAddressRequest extends AbstractObject
8  implements Serializable {
9
10     /**
11      * 订单id
12      */
13     private String orderId;
14
15     /**
16      * 省
17      */
18     private String province;
19
20     /**
21      * 市
22      */
23     private String city;
24
25     /**
26      * 区
27      */
28     private String area;
29
30     /**
31      * 街道地址
32      */
33     private String street;
34
35     /**
36      * 详细地址
37      */
38     private String detailAddress;
39
40     /**
41      * 经度
42      */
43     private BigDecimal lon;
44
45     /**
46      * 纬度
47      */
48     private BigDecimal lat;
```

返回结果

Java [复制代码](#)

```
1  /**
2   * 调整订单配送地址结果
3   * @author zhonghuashishan
4   * @version 1.0
5   */
6  @Data
7  public class AdjustDeliveryAddressDTO extends AbstractObject implements
    Serializable {
8
9      /**
10     * 响应结果
11     */
12     private boolean result;
13 }
```

业务规则

订单只有还未出库才可以修改地址，有且仅可以修改一次

具体实现

- 1、根据orderId查询出一笔订单出来，如果查询为null，直接抛异常
- 2、接着查询order_delivery_detail，如果查询为null，直接抛异常
- 3、校验之前是否已经修改过一次，如果是，直接抛异常
- 4、校验订单是否还未出库（10，20，30），如果已出库，直接抛异常
- 5、进行具体的业务逻辑：更新order_delivery_detail中的地址信息
- 6、构造返参，返回给调用方

1.11、查询订单列表

所属系统：订单系统(ruyuan-eshop-order-service)

实现方法

OrderQueryApi.listOrders()

接口定义

Java | [复制代码](#)

```
1  /**
2   * 查询订单列表
3   * @param query
4   * @return
5   */
6  JsonResult<PagingInfo<OrderListDT0>> listOrders(OrderQuery query);
```

请求参数

```
1  /**
2   * 订单列表查询请求
3   * @author zhonghuashishan
4   * @version 1.0
5   */
6  @Data
7  public class OrderQuery extends AbstractObject implements Serializable {
8
9      public static final Integer MAX_PAGE_SIZE = 100;
10
11     /**
12      * 业务线
13      */
14     private Integer businessIdentifier;
15     /**
16      * 订单类型
17      */
18     private Set<Integer> orderTypes;
19     /**
20      * 订单号
21      */
22     private Set<String> orderIds;
23     /**
24      * 卖家ID
25      */
26     private Set<String> sellerIds;
27     /**
28      * 父订单号
29      */
30     private Set<String> parentOrderIds;
31     /**
32      * 用户ID
33      */
34     private Set<String> userIds;
35     /**
36      * 订单状态
37      */
38     private Set<Integer> orderStatus;
39     /**
40      * 收货人手机号
41      */
42     private Set<String> receiverPhones;
43     /**
44      * 收货人姓名
45      */
46     private Set<String> receiverNames;
47     /**
48      * 交易流水号
```

```

49     */
50     private Set<String> tradeNos;
51
52     /**
53      * sku code
54      */
55     private Set<String> skuCodes;
56     /**
57      * sku商品名称
58      */
59     private Set<String> productNames;
60     /**
61      * 创建时间-查询区间
62      */
63     private Pair<Date,Date> createdTimeInterval;
64     /**
65      * 支付时间-查询区间
66      */
67     private Pair<Date,Date> payTimeInterval;
68     /**
69      * 支付金额-查询区间
70      */
71     private Pair<Integer,Integer> payAmountInterval;
72
73     /**
74      * 页码
75      */
76     private Integer pageNo = 1;
77     /**
78      * 每页条数
79      */
80     private Integer pageSize = 20;
81 }
82

```

返回结果

```
1  @Data
2  public class OrderListDTO extends AbstractObject implements Serializable {
3
4      /**
5       * 接入方业务线标识
6       */
7      private Integer businessIdentifier;
8
9      /**
10     * 订单编号
11     */
12     private String orderId;
13
14     /**
15     * 父订单编号
16     */
17     private String parentOrderId;
18
19     /**
20     * 接入方订单号
21     */
22     private String businessOrderId;
23
24     /**
25     * 订单类型
26     */
27     private Integer orderType;
28
29     /**
30     * 订单状态
31     */
32     private Integer orderStatus;
33
34     /**
35     * 卖家编号
36     */
37     private String sellerId;
38
39     /**
40     * 买家编号
41     */
42     private String userId;
43
44     /**
45     * 支付方式
46     */
47     private Integer payType;
48
```

```
49     /**
50      * 使用的优惠券编号
51      */
52     private String couponId;
53
54     /**
55      * 支付时间
56      */
57     private Date payTime;
58
59     /**
60      * 用户备注
61      */
62     private String userRemark;
63
64     /**
65      * 订单评论状态 0:未发表评论 1:已发表评论
66      */
67     private Integer commentStatus;
68
69
70     // order_item
71
72     /**
73      * 商品图片
74      */
75     private String productImg;
76
77     /**
78      * 商品名称
79      */
80     private String productName;
81
82     /**
83      * sku编码
84      */
85     private String skuCode;
86
87     /**
88      * 销售数量
89      */
90     private Integer saleQuantity;
91
92     /**
93      * 交易支付金额
94      */
95     private Integer payAmount;
96 }
```

业务规则

无效状态（255）订单不展示

字段映射梳理

1、入参和表字段映射关系梳理

- 业务线 businessIdentifier->order_info.business_identifier
- 订单类型 orderTypes -> order_info.order_type
- 订单号 orderIds -> order_info.order_id
- 卖家ID sellerIds -> order_info.seller_id
- 父订单号 parentOrderIds -> order_info.parent_order_id
- 订单状态 orderStatus -> order_info.order_status
- 创建时间-开始/截止 startCreatedTime, endCreatedTime -> order_info.gmt_create
- 支付时间-开始/截止 startPayTime, endPayTime -> order_info.pay_time
- 用户ID userIds -> order_info.user_id
- 支付金额-区间开始/区间截止 startPayAmount, endPayAmount -> order_info.pay_amount
- sku code skuCodes -> order_item.sku_code
- sku商品名称 productNames -> order_item.product_name
- 收货人手机号 receiverPhones -> order_delivery_detail.receiver_phone
- 收货人姓名 receiverPhones -> order_delivery_detail.receiver_name
- 交易流水号 tradeNos -> order_payment_detail.out_trade_no
- 下单平台 orderPlatforms -> order_channel.platform

2、返回结果和数据表字段映射关系梳理

- 业务线 businessIdentifier->order_info.business_identifier
- 订单号 orderId -> order_info.order_id
- 卖家ID sellerId -> order_info.seller_id
- 用户ID userId -> order_info.user_id
- 接入方订单ID businessOrderId -> order_info.business_order_id
- 父订单号 parentOrderId -> order_info.parent_order_id
- 订单状态 orderStatus -> order_info.order_status
- 支付方式 payTyoe -> order_info.pay_type
- 支付时间payTime -> order_info.pay_time

- 优惠券ID -> order_info.coupon_id
- 用户备注 userRemark -> order_info.user_remark
- 订单评论状态 commentStatus -> order_info.comment_status
- 商品图片productImg -> order_item.product_img
- 商品名称productName -> order_item.product_name
- sku code skuCode -> order_item.sku_code
- 销售数量 saleQuantity -> order_item.sale_quantity
- 支付金额payAmount -> order_item.pay_amount

具体实现：

1. 组装业务规则
2. 连表分页查询：order_info, order_item, order_delivery_detail, order_payment_detail, 具体的
字段映射如上方的入参和表字段映射关系梳理
3. 如果没有给排序字段，默认按照订单创建时间降序
4. 对于查询出来的每条记录，根据上面的返回结果和数据表字段映射关系梳理，查询对应数据表，组
装返参，返回给调用方

实现亮点：

1. 通过builder模式构建复杂查询业务规则

Java [复制代码](#)

```

1 //1、组装业务查询规则
2 OrderListQueryDTO queryDTO = OrderListQueryDTO.Builder.builder()
3     .copy(query)
4     //不展示无效订单
5     .removeInvalidStatus()
6     .setPage(query)
7     .build();
8
9 //2、查询
10 Page<OrderListDTO> page = orderInfoDAO.listByPage(queryDTO);

```

二期功能实现

- 1、可以通过参数控制返回字段

2、可以通过参数控制排序方式

Java [复制代码](#)

```
1 public JsonResult<List<OrderDetailDTO>> listOrders(OrderQuery
   qry, QueryDataTypeEnums... queryDataType)
```

Java [复制代码](#)

```
1 public enum OrderQueryDataTypeEnums {
2     /**订单基础**/
3     BASE,
4     /**订单条目**/
5     ORDER_ITEM,
6     /**订单费用明细**/
7     ORDER_AMOUNT_DETAIL,
8     /**订单支付**/
9     PAYMENT,
10    /**订单配送信息**/
11    DELIVERY,
12    /**售后**/
13    AFTER_SALE,
14    /**售后明细**/
15    AFTER_SALE_DETAIL,
16    /**售后交易**/
17    AFTER_SALE_PAY,
18    /**渠道**/
19    CHANNEL,
20    /**费用**/
21    AMOUNT,
22    /**操作日志**/
23    OPERATE_LOG,
24    /**订单快照**/
25    SNAPSHOT
26    ;
27 }
28
29 public enum OrderQuerySortField {
30     /**按照订单创建时间倒序**/
31     CREATE_TIME_DESC,
32     /**按照支付时间倒序**/
33     PAY_TIME_DESC,;
34 }
```

OrderQueryDataTypeEnums，出参字段，以及数据表映射关系梳理

OrderQueryDataTypeEnums	OrderDetailDTO	表
订单基础 BASE	OrderInfoDTO orderInfo	order_info
订单条目 ORDER_ITEM	List<OrderItemDTO> orderItems	order_item
订单费用明细 ORDER_AMOUNT_DETAIL	List<OrderAmountDetailDTO> orderAmountDetails	order_amount_detail
订单支付 PAYMENT	List<OrderPaymentDetailDTO> orderPaymentDetails	order_payment_detail
订单配送信息 DELIVERY	OrderDeliveryDetailDTO orderDeliveryDetail	order_delivery_detail
售后 AFTER_SALE	List<AfterSaleInfoDTO> afterSaleInfos	after_sale_info
售后明细 AFTER_SALE_DETAIL	Map<String,List<AfterSaleDetailDTO>> afterSaleDetails	after_sale_detail
售后交易 AFTER_SALE_PAY	Map<String, List<AfterSalePayDTO>> afterSalePays	after_sale_pay
费用 AMOUNT	Map<AmountTypeEnum, Integer> orderAmounts Map<Integer, Integer> orderAmountMap	order_amount
操作日志 OPERATE_LOG	List<OrderOperateLogDTO> orderOperateLogs	order_operate_log
订单快照 SNAPSHOT	List<OrderSnapshotDTO> orderSnapshots	order_snapshot

1.12、查询订单详情

所属系统：订单系统(ryuan-eshop-order-service)

实现方法

OrderQueryApi.orderDetail()

接口定义

Java | [复制代码](#)

```
1  /**
2   * 查询订单详情
3   * @param orderId
4   * @return
5   */
6  JsonResult<OrderDetailDTO> orderDetail(String orderId);
```

请求参数

Java | [复制代码](#)

```
1  orderId: 订单号
```

返回结果

```
1  /**
2   * <p>
3   *  订单详情DTO
4   * </p>
5   *
6   * @author zhonghuashishan
7   */
8  @Data
9  public class OrderDetailDTO {
10
11      /**
12       * 订单
13       */
14      private OrderInfoDTO orderInfo;
15      /**
16       * 订单条目
17       */
18      private List<OrderItemDTO> orderItems;
19      /**
20       * 订单费用明细
21       */
22      private List<OrderAmountDetailDTO> orderAmountDetails;
23      /**
24       * 订单配送信息表
25       */
26      private OrderDeliveryDetailDTO orderDeliveryDetail;
27      /**
28       * 订单支付明细
29       */
30      private List<OrderPaymentDetailDTO> orderPaymentDetails;
31
32      /**
33       * 费用类型
34       */
35      private Map<Integer, Integer> orderAmounts;
36
37      /**
38       * 订单操作日志
39       */
40      private List<OrderOperateLogDTO> orderOperateLogs;
41
42      /**
43       * 订单快照信息
44       */
45      private List<OrderSnapshotDTO> orderSnapshots;
46
47      /**
48       * 订单缺品信息
```

```

49      */
50      private List<OrderLackItemDTO> lackItems;
51  }

```

业务规则

除了展示一些正向订单相关信息外，还需要展示缺品时退单的金额信息（售后单）

字段映射梳理

OrderDetailDTO	表
OrderInfoDTO orderInfo	order_info
List<OrderItemDTO> orderItems	order_item
List<OrderAmountDetailDTO> orderAmountDetails	order_amount_detail
List<OrderPaymentDetailDTO> orderPaymentDetails	order_payment_detail
OrderDeliveryDetailDTO orderDeliveryDetail	order_delivery_detail
Map<Integer, Integer> orderAmountMap	order_amount
List<OrderOperateLogDTO> orderOperateLogs	order_operate_log
List<OrderSnapshotDTO> orderSnapshot	order_snapshot
List<OrderLackItemDTO> lackItems	after_sale_info,after_sale_item

具体实现

1. 根据orderId查询order_info表记录
2. 如果查询为null，直接返回null
3. 如果能查询到orderInfo记录，那么根据上面的**字段映射梳理**，组装返参
4. 缺品标识存储在order_info.ext_json字段里面

2、逆向售后业务

2.1、取消订单/超时未支付取消 接口

接口定义：

```
1      /**
2      * 取消订单/超时未支付取消 入口
3      * <p>
4      * 有3个调用的地方：
5      * 1、用户手动取消，订单出库状态之前都可以取消
6      * 2、消费正向生单之后的MQ取消，要先判断支付状态，未支付才取消。
7      * 3、定时任务定时扫描，超过30分钟，未支付才取消
8      */
9      JsonResult<Boolean> cancelOrder(CancelOrderRequest
cancelOrderRequest);
```

Java

 复制代码

请求入参：

```
1  @Data
2  public class CancelOrderRequest extends AbstractObject implements
   Serializable {
3
4      private static final long serialVersionUID = -4035579903997700971L;
5
6      /**
7       * 订单号
8       */
9      private String orderId;
10
11     /**
12      * 订单渠道来源
13      */
14     private Integer businessIdentifier;
15
16     /**
17      * 订单取消类型 0-手动取消 1-超时未支付
18      */
19     private Integer cancelType;
20
21     /**
22      * 用户id
23      */
24     private String userId;
25
26     /**
27      * 订单类型
28      */
29     private Integer orderType;
30
31     /**
32      * 订单状态
33      */
34     private Integer orderStatus;
35
36     /**
37      * 原订单状态
38      */
39     private Integer oldOrderStatus;
40
41 }
42
```

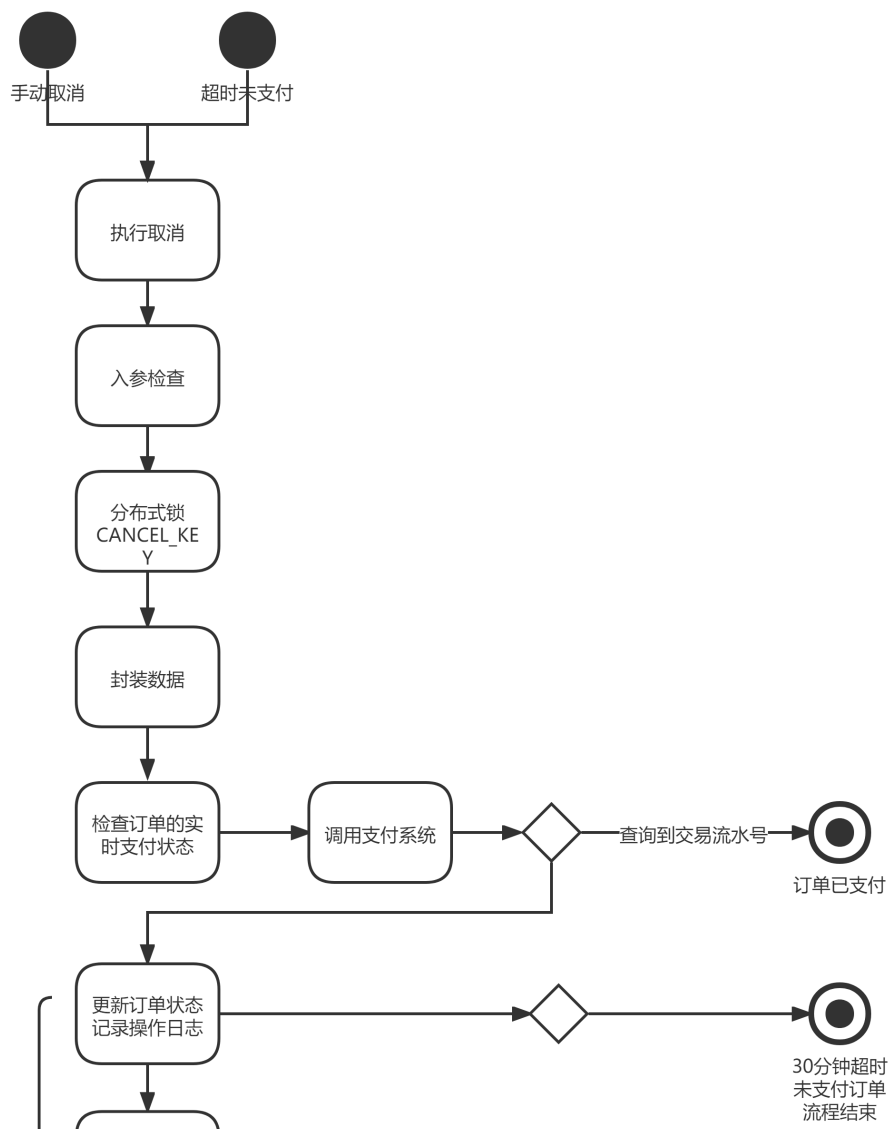
返回结果：

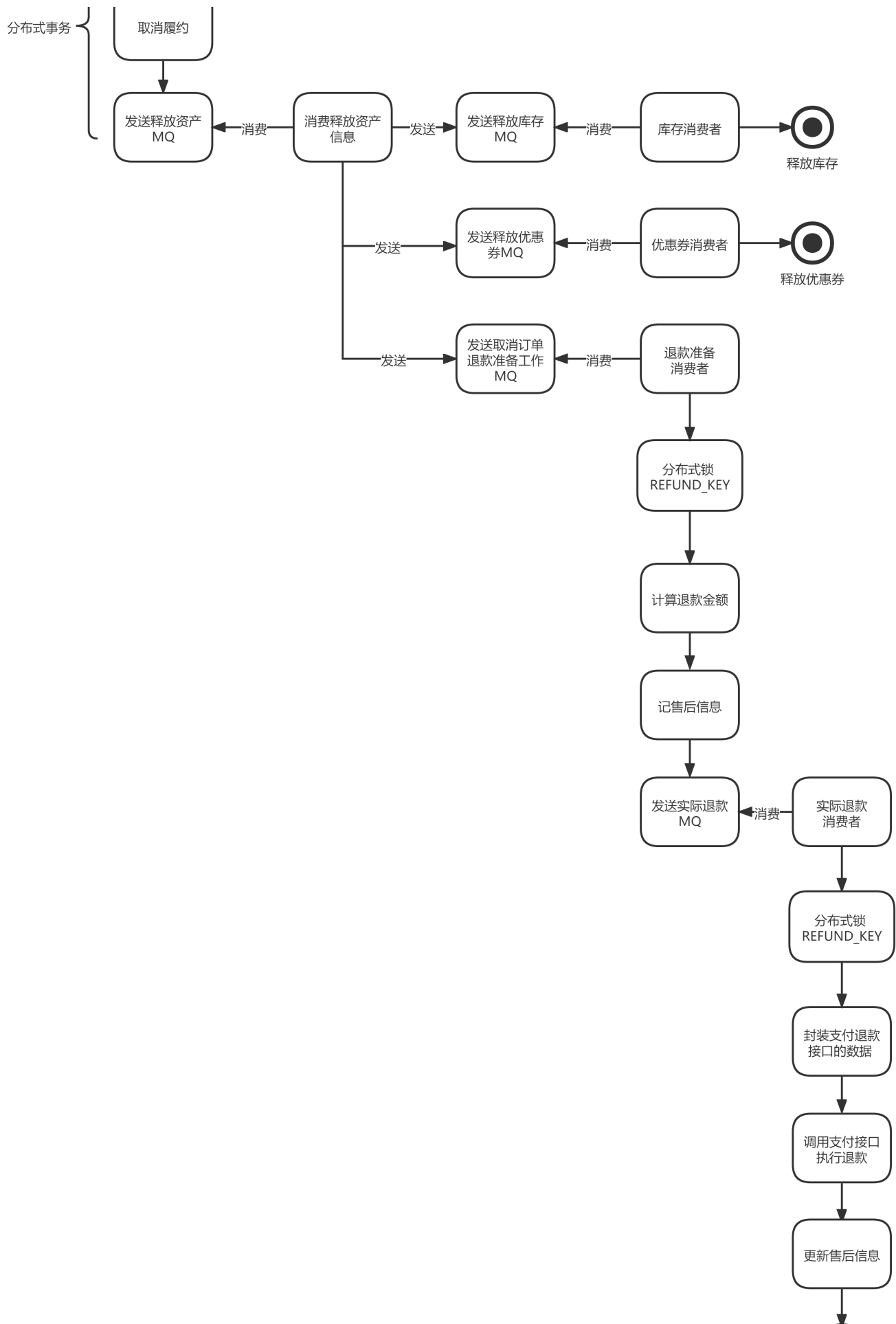

```
1  {  
2      "success": true,  
3      "data": null,  
4      "errorCode": null,  
5      "errorMessage": null  
6  }
```

使用场景：

1. 用户手动取消，订单出库状态之前都可以取消
2. 正向生单之后，将订单延迟30分钟发送消息MQ，30分钟后仍未支付则取消该订单
3. 定时任务定时扫描，超过30分钟，未支付才取消

具体实现：







结束

技术要点：

1. 更新订单状态、取消履约、发送释放资产是一个分布式事务，目前用seata框架解决
2. 超时订单处理方案：生成订单之后发送一个延迟消息，取消订单MQ的消费消息，然后调用取消订单的接口，延迟时间30分钟。

2.2、支付退款回调 接口

接口定义：

Java | [复制代码](#)

```
1      /**
2      * 取消订单支付退款回调 入口
3      */
4      JsonResult<Boolean> receivePaymentRefundCallback(RefundCallbackRequest
    payRefundCallbackRequest);
```

请求入参：

```
1  @Data
2  public class RefundCallbackRequest extends AbstractObject implements
    Serializable {
3      private static final long serialVersionUID = -5258136513569055820L;
4      /**
5       * 订单号
6       */
7      private String orderId;
8      /**
9       * 退款批次号
10     */
11     private String batchNo;
12     /**
13      * 支付接口返回来的退款结果状态 10 未退款 20 退款成功 30 退款失败
14      */
15     private Integer refundStatus;
16     /**
17      * 退款费用
18      */
19     private Integer refundFee;
20     /**
21      * 退款总额
22      */
23     private Integer totalFee;
24     /**
25      * 支付退款签名
26      */
27     private String sign;
28     /**
29      * 交易流水号
30      */
31     private String tradeNo;
32     /**
33      * 支付退款回调时间
34      */
35     private Date refundTime;
36     /**
37      * 订单售后id
38      */
39     private String afterSaleId;
40
41     /**
42      * 当前订单的售后退款状态
43      */
44     private Integer currentRefundStatus;
45 }
46
```

返回结果：

```
1  {
2      "success": true,
3      "data": null,
4      "errorCode": null,
5      "errorMessage": null
6  }
```

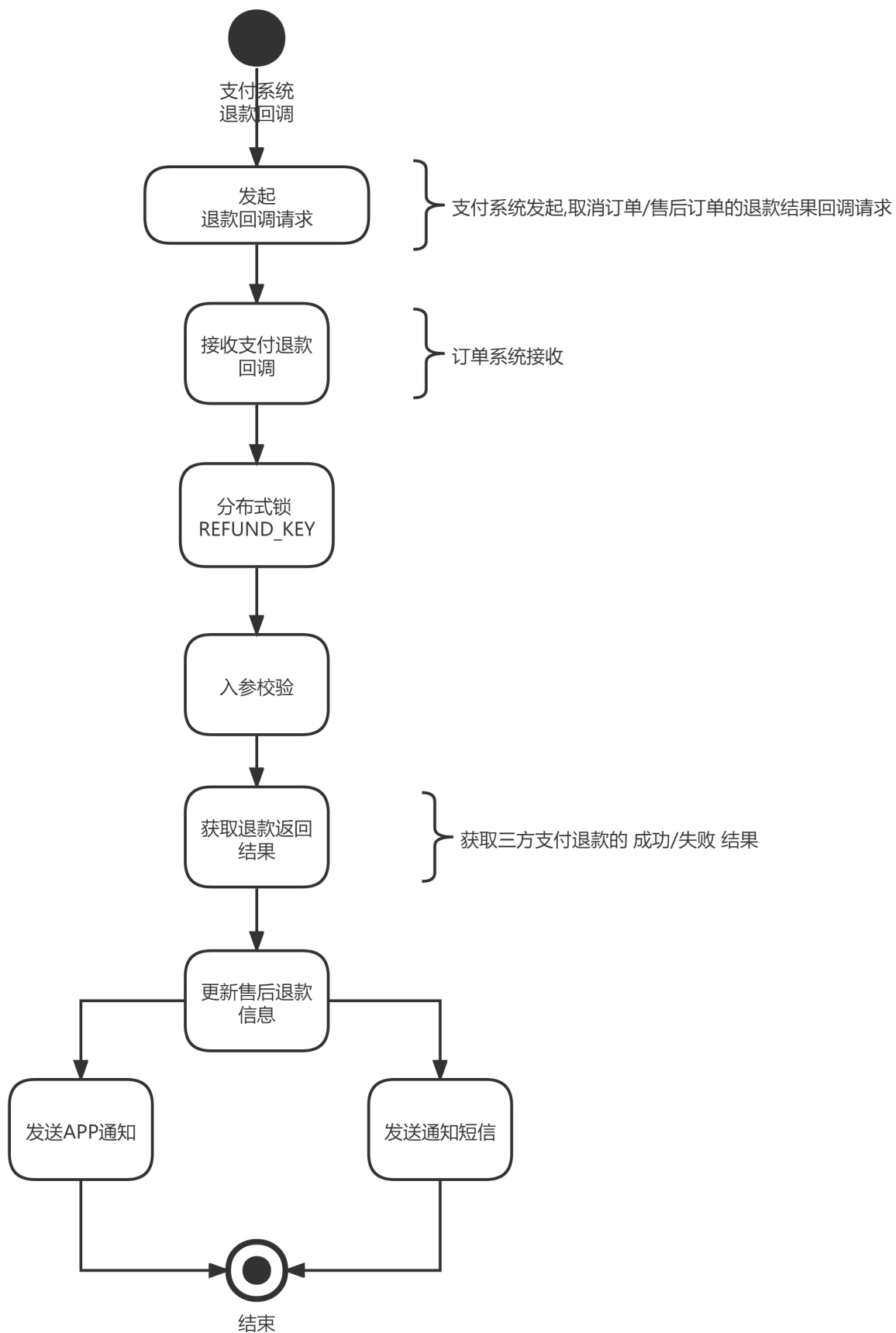
Java

 复制代码

使用场景：

1. 用户手动取消订单后并发起退款，经过一段时间之后，支付系统回调给订单系统退款结果

2. 结果包含：退款成功、退款



失败

具体实现：

2.3、处理售后申请 接口

接口定义：

```
1      /**
2      * 处理售后申请 入口
3      */
4      JsonResult<Boolean> processApplyAfterSale(ReturnGoodsOrderRequest
returnGoodsOrderRequest);
```

Java | [复制代码](#)

请求入参：

```
1  @Data
2  public class ReturnGoodsOrderRequest extends AbstractObject implements
    Serializable {
3      private static final long serialVersionUID = 4893844329777688246L;
4
5      /**
6       * 订单号
7       */
8      private String orderId;
9
10     /**
11      * 接入方业务线标识 1, "自营商城"
12      */
13     private Integer businessIdentifier;
14
15     /**
16      * 用户id
17      */
18     private String userId;
19
20     /**
21      * 退货原因选项
22      */
23     private Integer returnGoodsCode;
24
25     /**
26      * 退货原因说明
27      */
28     private String returnGoodsDesc;
29
30     /**
31      * 退货sku编号
32      */
33     private String skuCode;
34 }
35
```

返回结果：


```
1  {  
2      "success": true,  
3      "data": true,  
4      "errorCode": null,  
5      "errorMessage": null  
6  }
```

使用场景：

1. 售后申请接收的是用户发起的退货请求，退货分整笔退和单笔退
2. 目前订单格式：一笔订单对应多个条目，系统目前仅支持默认每次退货是退一个条目，不考虑部分退数量的场景
3. 例如：一笔订单包含两个条目，A条目商品数量是2，B条目商品数量是1，用户退货每次只退一个
 - a. 如果退A，不考虑A的商品数量，默认就是全退
 - b. 如果退B，同理
 - c. 如果已经先退掉了A，再次退B时发现本次退完之后，该笔订单的全部条目都已经退掉，则在B中补退运费，在B退成功以后回退给用户优惠券
4. 目前此业务场景的缺陷：
 - a. 不支持部分退，拆单退，目前实现判断是否是退货最后一单的逻辑比较简单
 - b. 最后一笔退货补退运费时，没有生成运费的售后单
 - i. 还是以上述例子为例：
 1. 先退掉A，生成一笔售后单，记录的实际退款金额是A的退款金额
 2. 再退掉B，此时B的实际退款金额 = 运费 + B的退款金额，生成一笔售后单
 3. 改造点：单独再生成一张运费的售后单

具体实现：







2.4、客服售后审核 接口

接口定义：

```
1  /**
2   * 接收客服审核结果 入口
3   */
4  JsonResult<Boolean>
    receiveCustomerAuditResult(CustomerReviewReturnGoodsRequest
        customerReviewReturnGoodsRequest);
```

Java | [复制代码](#)

请求入参：

```
1  @Data
2  public class CustomerReviewReturnGoodsRequest extends AbstractObject
   implements Serializable {
3
4      private static final long serialVersionUID = 5541950604615013941L;
5      /**
6       * 售后id
7       */
8      private Long afterSaleId;
9      /**
10     * 客服id
11     */
12     private String customerId;
13     /**
14     * 审核结果 1 审核通过 2 审核拒绝
15     */
16     private Integer auditResult;
17     /**
18     * 售后支付单id
19     */
20     private Long afterSaleRefundId;
21     /**
22     * 订单id
23     */
24     private String orderId;
25     /**
26     * 客服审核结果描述信息
27     */
28     private String auditResultDesc;
29
30 }
```

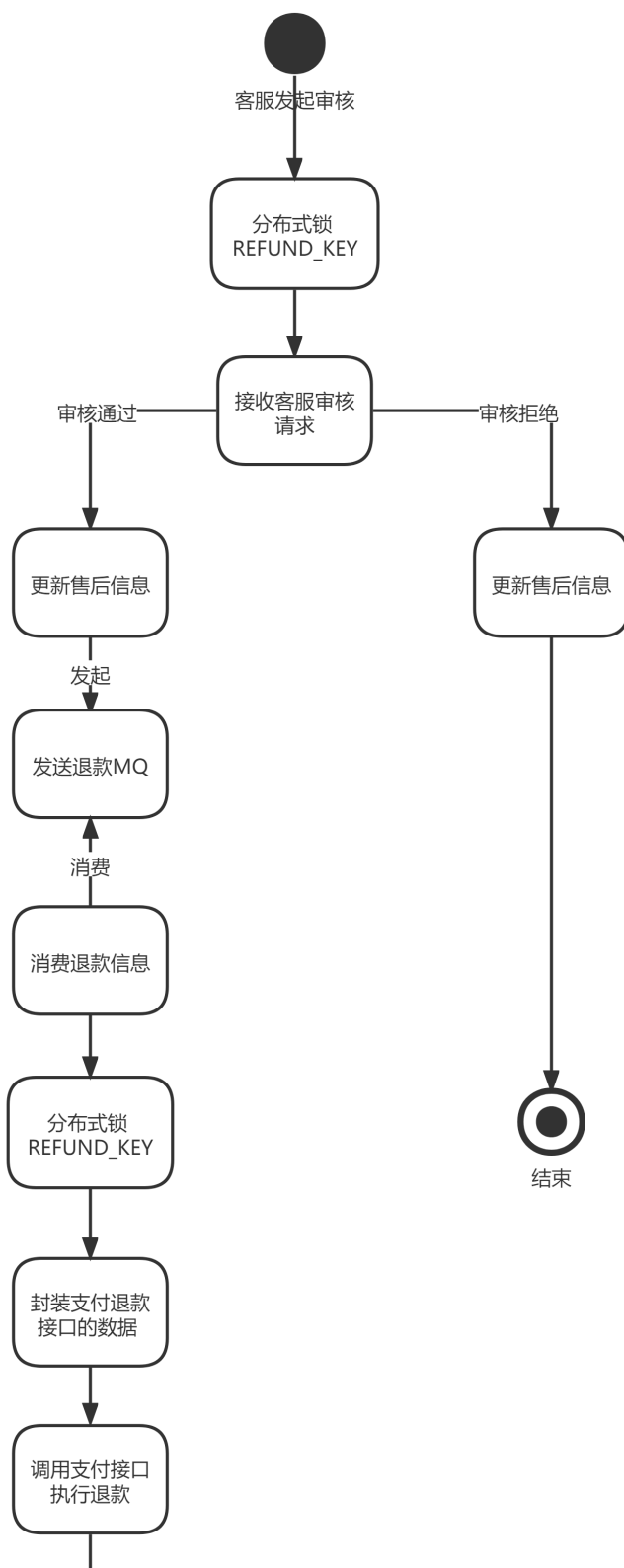
返回结果:

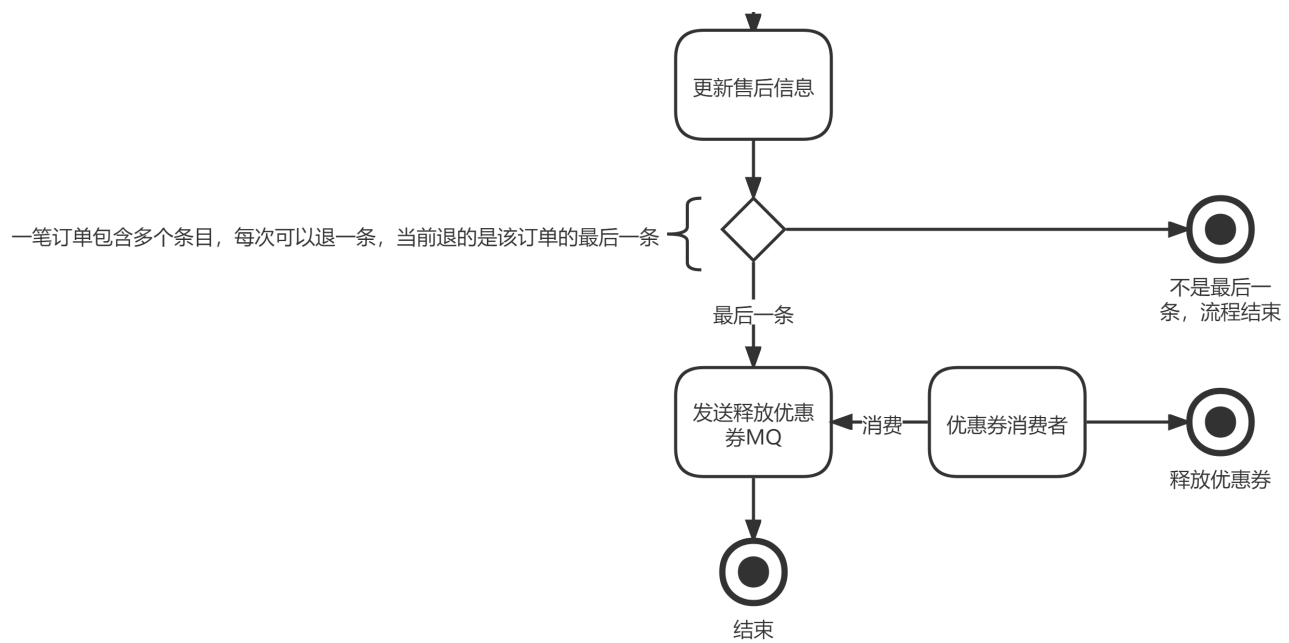
```
1  {
2      "success": true,
3      "data": true,
4      "errorCode": null,
5      "errorMessage": null
6  }
```

使用场景:

1. 客服审核用户的售后请求，分为审核通过和审核拒绝
2. 审核通过后，自动发起退款

具体实现：





2.5、缺品接口

所属系统：订单系统(ruyuan-eshop-order-service)

实现方法

OrderLackService.executeLackRequest()

接口定义

```
1  public interface OrderLackService {
2
3      /**
4       * 校验入参
5       * @param request
6       */
7      CheckLackDTO checkRequest(LackRequest request) throws
      OrderBizException;
8
9      /**
10     * 具体的缺品处理
11     * @param request
12     * @param checkLackDTO
13     * @return
14     */
15     LackDTO executeLackRequest(LackRequest request, CheckLackDTO
      checkLackDTO);
16 }
```

请求参数

```
1  /**
2   * 订单缺品请求
3   * @author zhonghuashishan
4   * @version 1.0
5   */
6  @Data
7  public class LackRequest extends AbstractObject implements Serializable {
8
9      /**
10       * 订单号
11       */
12     private String orderId;
13
14     /**
15      * 用户id
16      */
17     private String userId;
18
19     /**
20      * 具体的缺品项
21      */
22     private Set<LackItemRequest> lackItems;
23 }
24
25 /**
26 * 具体的缺品项
27 * @author zhonghuashishan
28 * @version 1.0
29 */
30 @Data
31 @AllArgsConstructor
32 @NoArgsConstructor
33 public class LackItemRequest extends AbstractObject implements
34 Serializable {
35
36     /**
37      * sku编码
38      */
39     private String skuCode;
40
41     /**
42      * 缺品数量
43      */
44     private Integer lackNum;
45
46     @Override
47     public boolean equals(Object o) {
```



```

48         if (this == o) return true;
49         if (o == null || getClass() != o.getClass()) return false;
50         LackItemRequest that = (LackItemRequest) o;
51         return Objects.equals(skuCode, that.skuCode);
52     }
53
54     @Override
55     public int hashCode() {
56         return Objects.hash(skuCode);
57     }
58 }
59

```

返回结果

```

1  @Data
2  public class LackDTO implements Serializable {
3      /**
4       * 订单id
5       */
6      private String orderId;
7      /**
8       * 售后id
9       */
10     private Long afterSaleId;
11 }
12

```

Java [复制代码](#)

使用场景

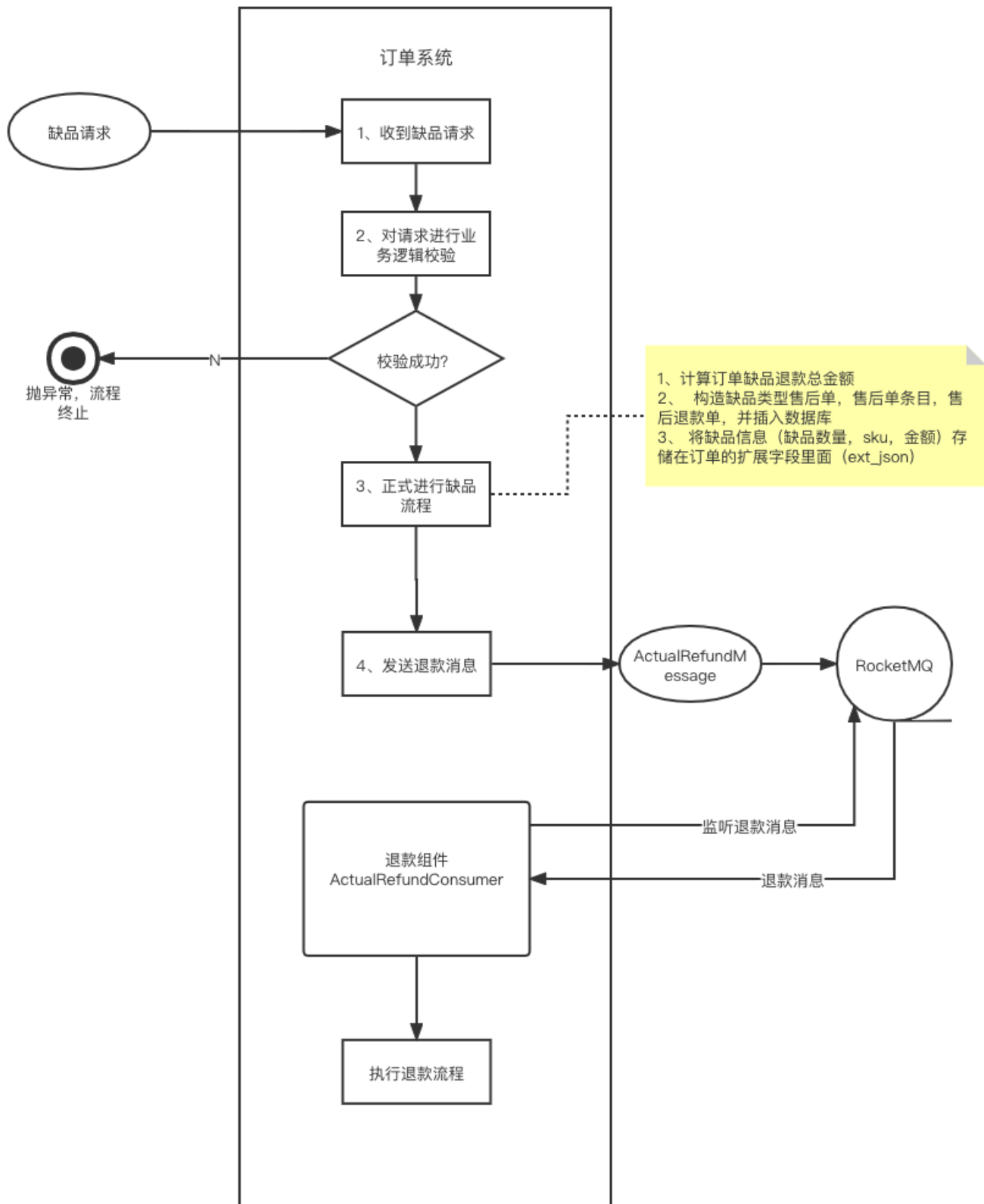
当订单履约后，被推送到履约系统，捡货人员根据订单要求拣货时，发现有商品缺货，那么就会直接对订单进行缺品退款，然后缺品部分的钱直接回退到用户账户去

业务规则

- 1、缺品的商品sku的数量不能超过下订单商品sku数量
- 2、只有支付之后，配送之前才可以发起缺品

具体实现

1. 订单系统收到缺品请求
2. 首先对缺品请求进行业务规则校验，校验失败直接抛异常
3. 校验成功进入缺品退款处理流程
 - a. 计算订单缺品退款总金额 = 每个缺品sku的缺品退款金额总和
 - i. 单个缺品sku的缺品退款金额计算公式 = (缺品数量/下单数量) * 原付款金额
 - b. 构造缺品类型售后单，售后单条目，售后退款单，并插入数据库
 - c. 将缺品信息（缺品数量，sku，金额）存储在订单的扩展字段里面（ext_json）
4. 发送退款消息（ActualRefundMessage）
5. 订单系统的退款组件(ActualRefundConsumer)收到退款消息后就会发起退款流程，将钱退还给用户



技术要点

1. 本地事务和发送mq消息的最终一致性，二期实现

2. 接口幂等：同样的缺品请求发送多次不会生成多条缺品记录，通过分布式锁+前置校验实现，二期实现

2.7、用户撤销售后申请

所属系统：订单系统(ryuan-eshop-order-service)

实现方法

AfterSaleApi.revokeAfterSale()

接口定义

```
1  /**
2   * 用户撤销售后申请
3   */
4   JsonResult<Boolean> revokeAfterSale(RevokeAfterSaleRequest request);
```

Java | [复制代码](#)

请求参数

```
1  @Data
2  public class RevokeAfterSaleRequest {
3      /**
4       * 售后单
5       */
6      private Long afterSaleId;
7  }
```

Java | [复制代码](#)

返回结果

```
1 true/false
```

业务规则

只有提交申请状态才可以撤销

具体实现

1. 接受撤销售后申请请求
2. 查询售后单
3. 进行业务规则判断
4. 加refund_key的分布式锁
5. 更新售后单状态为：“已撤销”
6. 增加一条售后单操作日志
7. 释放锁

技术要点

1. 加refund_key的分布式锁：只要涉及售后表的更新，就需要加锁，锁整个售后表，否则算钱的时候，就会由于突然撤销，导致钱多算了

2.8、自动取消超时订单任务

所属系统：订单系统(ruyuan-eshop-order-service)

实现方法

AutoCancelExpiredOrderTask

使用场景

后台定时任务每间隔一定时间就会扫描系统中超过30min未支付的订单对其进行取消

具体实现

1. spring schedule启动一个调度任务，每隔30min运行一次，扫描系统中所有状态是“未支付”的订单
2. 对每一条订单进行超时判断：
 - a. 只要 $\text{currentTime} - \text{expireTime} \geq 30\text{min}$ ，那么就认为这笔订单已经超时，需要取消
3. 对需要取消的订单调用订单取消结果，见上面的2.1、取消订单/超时未支付取消接口

技术要点

- 1、如何保证不会扫描太多的订单：只扫描一定时间间隔内的未支付订单，二期实现
- 2、二期换成xxxjob来实现

2.9、查询售后列表

所属系统：订单系统(ryuan-eshop-order-service)

实现方法

AfterSaleQueryApi.listAfterSales()

接口定义

```
1  /**
2   * 查询售后列表
3   *
4   * @param query
5   * @return
6   */
7  JsonResult<PagingInfo<AfterSaleOrderListDT0>>
  listAfterSales(AfterSaleQuery query);
```

Java | [复制代码](#)

请求参数

```
1  @Data
2  public class AfterSaleQuery extends AbstractObject implements Serializable
3  {
4      public static final Integer MAX_PAGE_SIZE = 100;
5
6      /**
7       * 业务线
8       */
9      private Integer businessIdentifier;
10
11     /**
12      * 订单类型
13      */
14     private Set<Integer> orderTypes;
15     /**
16      * 售后单状态
17      */
18     private Set<Integer> afterSaleStatus;
19     /**
20      * 售后申请来源
21      */
22     private Set<Integer> applySources;
23     /**
24      * 售后类型
25      */
26     private Set<Integer> afterSaleTypes;
27     /**
28      * 售后单号
29      */
30     private Set<Long> afterSaleIds;
31     /**
32      * 订单号
33      */
34     private Set<String> orderIds;
35     /**
36      * 用户ID
37      */
38     private Set<String> userIds;
39     /**
40      * sku code
41      */
42     private Set<String> skuCodes;
43     /**
44      * 创建时间-查询区间
45      */
46     private Pair<Date,Date> createdTimeInterval;
47     /**
```



```
48      * 售后申请时间-查询区间
49      */
50     private Pair<Date,Date> applyTimeInterval;
51     /**
52      * 售后客服审核时间-查询区间
53      */
54     private Pair<Date,Date> reviewTimeInterval;
55     /**
56      * 退款支付时间-查询区间
57      */
58     private Pair<Date,Date> refundPayTimeInterval;
59     /**
60      * 退款金额-查询区间
61      */
62     private Pair<Integer,Integer> refundAmountInterval;
63     /**
64      * 页码；默认为1；
65      */
66     private Integer pageNo = 1;
67     /**
68      * 一页的数据量。默认为20
69      */
70     private Integer pageSize = 20;
71
72 }
```

返回结果

```
1  @Data
2  public class AfterSaleOrderListDTO extends AbstractObject implements
   Serializable {
3
4      /**
5       * 接入方业务标识
6       */
7      private Integer businessIdentifier;
8      /**
9       * 售后单号
10     */
11     private String afterSaleId;
12     /**
13      * 订单号
14      */
15     private String orderId;
16     /**
17      * 订单类型
18      */
19     private Integer orderType;
20     /**
21      * 买家编号
22      */
23     private String userId;
24     /**
25      * 订单实付金额
26      */
27     private Integer orderAmount;
28     /**
29      * 售后单状态
30      */
31     private Integer afterSaleStatus;
32     /**
33      * 申请售后来源
34      */
35     private Integer applySource;
36     /**
37      * 申请售后时间
38      */
39     private Date applyTime;
40     /**
41      * 申请理由编码
42      */
43     private Integer applyReasonCode;
44     /**
45      * 申请理由描述
46      */
47     private String applyReason;
```

```
48
49     /**
50      * 客服审核时间
51      */
52     private Date reviewTime;
53     /**
54      * 客服审核结果编码
55      */
56     private Long reviewReasonCode;
57     /**
58      * 客服审核结果
59      */
60     private String reviewReason;
61
62     /**
63      * 备注
64      */
65     private String remark;
66
67     /**
68      * 售后类型
69      */
70     private Integer afterSaleType;
71
72     /**
73      * 售后类型详情
74      */
75     private Integer afterSaleTypeDetail;
76
77     // after_sale_detail
78
79     /**
80      * sku code
81      */
82     private String skuCode;
83
84     /**
85      * 商品名
86      */
87     private String productName;
88
89     /**
90      * 商品图片地址
91      */
92     private String productImg;
93
94     /**
95      * 商品退货数量
96      */
97     private Integer returnQuantity;
```

```

98
99     /**
100     * 商品总金额
101     */
102     private Integer originAmount;
103
104     /**
105     * 申请退款金额
106     */
107     private Integer applyRefundAmount;
108
109     /**
110     * 实际退款金额
111     */
112     private Integer realRefundAmount;
113 }

```

业务规则

售后列表和详情只能看到用户主动发起的售后退款记录，超时自动取消和用户手动取消的售后单不展示，通过售后单的类型售后类型来区分

字段映射梳理

1、入参和表字段映射关系梳理

- 业务线 businessIdentifier -> after_sale_info.business_identifier
- 订单类型 orderTypes -> after_sale_info.order_type
- 售后单号 afterSaleIds -> after_sale_info.order_id
- 订单号 orderIds -> after_sale_info.order_id
- 用户ID userIds -> after_sale_info.user_id
- 售后单状态 afterSaleStatus -> after_sale_info.after_sale_status
- 售后申请来源 applySources -> after_sale_info.apply_source
- 售后类型 afterSaleTypes -> after_sale_info.after_sale_type
- 售后申请时间-开始/截止 startApplyTime, endApplyTime -> after_sale_info.apply_time
- 售后客户审核时间-开始/截止 startReviewTime, endReviewTime -> after_sale_info.review_time
- 退款支付时间-开始/截止 startRefundPayTime, endRefundPayTime -> after_sale_pay.refund_pay_time

- 创建时间-开始/截止 startCreatedTime, endCreatedTime -> after_sale_pay.gmt_create
- 退款金额-区间开始/区间截止 startRefundAmount,endRefundAmount -> after_sale_pay.refund_amount
- skuCodes -> after_sale_detail.sku_id

2、返回结果和数据表字段映射关系梳理

- 业务线 businessIdentifier->after_sale_info.business_identifier
- 售后单ID afterSaleId -> after_sale_info.after_sale_id
- 订单ID orderId -> after_sale_info.order_id
- 订单类型 orderType -> after_sale_info.order_type
- 买家ID userId-> after_sale_info.user_id
- 售后单状态 orderStatus -> after_sale_info.after_sale_status
- 申请售后来源 applySource -> after_sale_info.apply_source
- 申请时间 applyTime -> after_sale_info.apply_time
- 申请原因编码 applyReasonCode -> after_sale_info.apply_reason_code
- 申请原因描述 applyReason -> after_sale_info.apply_reason
- 客服审核时间 reviewTime -> after_sale_info.review_time
- 客服审核结果编码 reviewReasonCode -> after_sale_info.review_reason_code
- 客服审核结果 reviewReason -> after_sale_info.review_reason
- 备注 remark -> after_sale_info.remark
- 售后类型 afterSaleType -> after_sale_info.after_sale_type
- 售后类型详情 afterSaleTypeDetail -> after_sale_info.after_sale_type_detail
- sku code skuCode -> after_sale_item.sku_code
- 商品名称 productName -> after_sale_item.product_name
- 商品图片 productImg -> after_sale_item.product_img
- 商品退货数量 returnQuantity -> after_sale_item.return_quantity
- 商品总金额 originAmount -> after_sale_item.origin_amount
- 申请退款金额applyRefundAmount -> after_sale_item.apply_refund_amount
- 实际退款金额realRefundAmount -> after_sale_item.real_refund_amount

具体实现

1. 组装业务查询规则
2. 连表分页查询: after_sale_info, after_sale_detail, after_sale_pay, 具体的字段映射如上方的入参和表字段映射关系梳理
3. 如果没有给排序字段, 默认按照售后单创建时间降序

4. 查询出来的每条记录，根据上面的返回结果和数据表字段映射关系梳理，查询对应数据表，组装返回，返回给调用方。
5. 通过builder模式构建复杂查询业务规则

Java | [复制代码](#)

```
1 //1、组装业务查询规则
2 AfterSaleListQueryDTO queryDTO = AfterSaleListQueryDTO.Builder.builder()
3     .copy(query)
4     //默认只展示用户主动发起的售后单
5     .userApplySource()
6     .setPage(query)
7     .build();
8
9
10 //2、查询
11 Page<AfterSaleOrderListDTO> page = afterSaleInfoDAO.listByPage(queryDTO);
```

2.10、查询售后详情

所属系统：订单系统(ruyuan-eshop-order-service)

实现方法

AfterSaleQueryApi.listAfterSales()

接口定义

Java | [复制代码](#)

```
1 /**
2  * 查询售后单详情
3  * @param afterSaleId
4  * @return
5  */
6 AfterSaleOrderDetailDTO afterSaleDetail(Long afterSaleId);
```

请求参数

1 afterSaleId: 售后单号

Java | [复制代码](#)

返回结果

```
1  @Data
2  public class AfterSaleOrderDetailDTO implements Serializable {
3
4      /**
5       * 售后信息
6       */
7      private AfterSaleInfoDTO afterSaleInfo;
8
9      /**
10     * 售后单条目
11     */
12     private List<AfterSaleItemDTO> afterSaleItems;
13
14     /**
15     * 售后支付信息
16     */
17     private List<AfterSalePayDTO> afterSalePays;
18
19     /**
20     * 售后单日志
21     */
22     private List<AfterSaleLogDTO> afterSaleLogs;
23 }
```

Java | [复制代码](#)

字段映射梳理

OrderDetailDTO	表
List<AfterSaleInfoDTO> afterSaleInfos	after_sale_info
List<AfterSaleItemDTO> afterSaleItems;	after_sale_item
List<AfterSalePayDTO> afterSalePays	after_sale_pay
List<AfterSaleLogDTO> afterSaleLogs	after_sale_log

具体实现

1. 根据afterSaleId查询after_sale_info表记录
2. 如果查询为null，直接返回null
3. 如果能查询到售后单记录，那么根据上方的**字段映射梳理**，组装返参，返回给调用方